LicoFast

Sistema web de e-commerce con POS, inventario, pagos y delivery

Documento de Diseño de Software

Arquitectura · Requisitos · Historias de usuario · UML

EQUIPO — GRUPO N1

José Luis Bayo Mora
Daniel Beltrán López
Fidel Cristian Castro Choque

DOCUMENTO

Versión 1.0
31 de octubre de 2026
Estado: para revisión

Facultad de Tecnología · Ingeniería de Sistemas · Universidad San Francisco Xavier de Chuquisaca

Contenido

Contenido 2

1. Introducción 4

1.1 Propósito del documento 4

1.2 Alcance 4

1.3 Definiciones y acrónimos 5

1.4 Referencias 5

2. Contexto y visión general 6

2.1 Problema y oportunidad 6

2.2 Partes interesadas y roles del sistema 6

2.3 Estado actual del proyecto (análisis del código) 6

3. Requisitos 8

3.1 Enfoque 8

3.2 Requisitos funcionales 8

3.3 Requisitos no funcionales 9

3.4 Restricciones y diferencias entre la Visión y el código 10

3.5 Trazabilidad requisito → módulo 10

4. Historias de usuario 11

4.1 Módulo de Seguridad 11

4.2 Módulo de E-commerce 12

4.3 Módulo de Pagos 14

4.4 Módulo de Distribución 16

5. Especificación de casos de uso 18

5.1 Actores 18

5.2 Especificaciones detalladas 19

6. Pila de desarrollo (stack tecnológico) 24

7. Arquitectura de software 25

7.1 Fundamentos 25

7.2 Factores que condicionan la arquitectura 26

7.3 Estilos arquitectónicos evaluados 26

7.4 Estilo adoptado: servicio REST en capas, con módulos 27

7.5 Descomposición modular y dependencias 28

7.6 Módulo Seguridad . . . 29
7.7 Módulo E-commerce . . . 30
7.8 Módulo Pagos . . . 32
7.9 Módulo Distribución . . . 34
7.10 Diseño de la API REST . . . 35
7.11 Vista de datos . . . 36
7.12 Vista de comportamiento . . . 37
7.13 Patrones de diseño aplicados . . . 37
7.14 Arquitectura de seguridad . . . 38
7.15 Escenarios de atributos de calidad . . . 39
7.16 Decisiones arquitectónicas (ADR) . . . 39
7.17 Análisis del código actual frente a la arquitectura . . . 40

8. Modelado UML . . . 42
8.1 Diagramas de casos de uso . . . 42
8.2 Diagramas de clases . . . 45
8.3 Diagrama de componentes . . . 49
8.4 Diagramas de despliegue . . . 50
8.5 Modelo entidad-relación . . . 52
8.6 Diagramas de secuencia . . . 53
8.7 Diagramas de estados . . . 55

9. Plan de implementación, pruebas y riesgos . . . 56
9.1 Iteraciones . . . 56
9.2 Estrategia de pruebas . . . 56
9.3 Riesgos . . . 57
9.4 Matriz de trazabilidad . . . 57

Anexo A. Glosario . . . 59

Anexo B. Esquema de datos propuesto (DDL) . . . 60

Anexo C. Del código actual a la arquitectura objetivo . . . 63

Anexo D. Referencias . . . 64

Anexo E. Implementación de referencia (LicoFastFinal) . . . 65
E.1 Estructura . . . 65
E.2 Endpoints añadidos respecto a la sección 7 . . . 65
E.3 Estado de los hallazgos de la sección 7.17 . . . 66

1. Introducción

1.1 Propósito del documento

Este **Documento de Diseño de Software (SDD)** define la arquitectura y el diseño del sistema **LicoFast**, una plataforma web de comercio electrónico con Punto de Venta (POS), control de inventario, cobranza y delivery para una licorería. El documento sirve como referencia común del equipo de desarrollo: traduce la Visión del Proyecto en requisitos verificables, historias de usuario, decisiones de arquitectura y modelos UML que se pueden implementar y probar.

El énfasis está en la **arquitectura de software**. Siguiendo el material de la asignatura (*Arquitectura de Software*, A. R. Durán Quiroga), la arquitectura se entiende como la estructura de mayor nivel del diseño: los componentes del sistema, sus relaciones y los principios que orientan su evolución. Por eso el SDD justifica cada decisión estructural con los atributos de calidad que la motivan (extensión, cambio, simplicidad y eficiencia) y con la evidencia del código ya construido.

1.2 Alcance

LicoFast se desarrolla para uso interno de una licorería específica. El sistema comparte un mismo catálogo e inventario entre la venta presencial (POS) y la venta online (web y, en una fase posterior, WhatsApp), y cubre el ciclo completo del comercio electrónico: catalogación, inventario, venta, salida del producto, distribución y cobranza. Se estructura en **cuatro módulos** expuestos mediante un **Web Service con API REST**:

Módulo	Responsabilidad principal	Origen en la Visión del Proyecto
Seguridad	Cuentas, autenticación, roles y permisos, verificación de edad, auditoría y respaldos.	Módulo de Seguridad (RF-16 a RF-22, RNF-01 a RNF-06).
E-commerce	Catálogo, carrito, pedidos, POS, inventario y stock, alertas, promociones y clientes frecuentes.	Módulo de E-commerce + Módulo de Almacenes + promociones (RF-01 a RF-03, RF-05 a RF-08, RF-14, RF-15).
Pagos	Cobranza en efectivo y por QR, caja, comprobantes y anulaciones.	Módulo de Finanzas (RF-04, RF-12, RF-13).
Distribución	Entregas a domicilio, asignación al repartidor, seguimiento y notificaciones.	Módulo de Distribución (RF-09 a RF-11).

> **Decisión de agrupación de módulos**
> La Visión del Proyecto define cinco módulos (E-commerce, Distribución, Finanzas, Seguridad y Almacenes). Para este SDD se pidió una arquitectura de **cuatro módulos**. Se hicieron dos fusiones explícitas: **Finanzas se llama Pagos** (caja, cobranza y facturación) y **Almacenes se integra en E-commerce**, porque inventario, catálogo y POS comparten las mismas tablas y reglas de stock. Las promociones (listadas en Finanzas en la Visión) se ubican en E-commerce porque se aplican al armar el carrito o la venta.

Fuera de alcance (primera versión): venta por otras redes sociales distintas de WhatsApp, integración con marketplaces de delivery (por ejemplo PedidosYa) y soporte multi-sucursal o multi-negocio.

1.3 Definiciones y acrónimos

Término	Definición
SDD	Software Design Document: documento que describe el diseño y la arquitectura del sistema.
API REST	Interfaz de programación basada en recursos HTTP identificados por URL, con operaciones estándar (GET, POST, PUT, DELETE) y sin estado en el servidor.
Web Service	Servicio accesible por red mediante protocolos web; en LicoFast, la API REST que consumen el frontend y otros clientes.
POS	Point of Sale: módulo de registro y cobro de ventas presenciales en el local.
SPA	Single Page Application: aplicación web de una sola página que consume la API por JavaScript.
RBAC	Role-Based Access Control: control de acceso según el rol del usuario.
JWT	JSON Web Token (RFC 7519): token firmado que transporta la identidad y el rol del usuario.
PBKDF2	Función de derivación de claves (RFC 8018) usada hoy para cifrar contraseñas con sal e iteraciones.
SAS / QR	Servicio del Banco Económico para generar y cobrar códigos QR (restricción de la Visión).
MVC	Modelo-Vista-Controlador: patrón usado por las vistas PHP del proyecto actual.
CU / HU / RF / RNF	Caso de uso / Historia de usuario / Requisito funcional / Requisito no funcional.
ADR	Architecture Decision Record: registro de una decisión de arquitectura con alternativas y consecuencias.

1.4 Referencias

- *Visión del Proyecto LicoFast*, Grupo N1, 2026 (documento base de requisitos de alto nivel).
- Durán Quiroga, A. R. *Arquitectura de Software*. Ingeniería de Software, USFX, 2019 (material de clase).
- Durán Quiroga, A. R. *Ingeniería de Requerimientos* (Capítulo 5, SIS-324). Sucre, 2006.
- Durán Quiroga, A. R. *Análisis* (Capítulo 6): modelo de análisis, clases de análisis y realización de casos de uso.
- Letelier Torres, P. *Desarrollo de Software Orientado a Objetos usando UML*: diagramas de casos de uso, de interacción y de clases. Universidad Politécnica de Valencia.
- Shaw, M. y Garlan, D. *Software Architecture: Perspectives on an Emerging Discipline*. Prentice Hall, 1996.
- IEEE Std 1016-2009 (*Software Design Descriptions*) e ISO/IEC/IEEE 42010:2011 (descripción de arquitecturas; sucesor de IEEE 1471-2000, citado en el material de clase).
- Fielding, R. *Architectural Styles and the Design of Network-based Software Architectures*. Tesis doctoral, UC Irvine, 2000.
- Fowler, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002. Gamma, E. et al. *Design Patterns*. Addison-Wesley, 1994.
- RFC 7519 (JSON Web Token) y RFC 8018 (PKCS #5, PBKDF2). OWASP Top 10 y OWASP Password Storage Cheat Sheet.
- Repositorio del proyecto: github.com/FCCASTROCH/Licofast (código analizado para este documento).

2. Contexto y visión general

2.1 Problema y oportunidad

La licorería opera hoy con registros manuales de ventas e inventario. Esto provoca pérdida de control sobre el stock real, errores de cobro, desorden en los pedidos recibidos por WhatsApp y falta de información para decidir qué reponer o cuánto ingresó en caja. En la zona existe poca oferta de delivery ágil y confiable, lo que representa una oportunidad de captar y fidelizar clientes con un sistema que integre venta, inventario, pedidos y entregas.

2.2 Partes interesadas y roles del sistema

El sistema define cinco roles de acceso, que coinciden con el campo `usuarios.rol` de la base de datos. El rol «empleado» del código actual se interpreta como el **encargado de bodega/inventario** de la Visión.

Rol en el sistema	Rol en la Visión	Dispositivo	Uso principal	Módulos
administrador	Dueño / Administrador del sistema	PC / tablet	Control total: usuarios, roles, inventario, caja, promociones, reportes.	SEG · ECO · PAG · DIS
cajero	Cajero / Vendedor	PC de caja	Registra ventas de mostrador y cobra (efectivo / QR); maneja la caja.	ECO · PAG
empleado (bodega)	Encargado de bodega	Tablet	Entradas y salidas de producto; conteo de existencias; alertas.	ECO
repartidor	Repartidor (delivery)	Celular	Recibe pedidos, entrega y registra el resultado y el cobro.	DIS · PAG
cliente	Cliente final	Celular / PC	Consulta el catálogo, compra, paga y sigue su pedido.	ECO · PAG · DIS
(visitante)	—	Cualquiera	Ve el catálogo y crea una cuenta.	SEG · ECO

2.3 Estado actual del proyecto (análisis del código)

Se analizó el repositorio del equipo para que el diseño parta de lo ya construido y no desde cero. El proyecto tiene una **aplicación web en PHP con vistas por rol** (administrador, cajero, cliente y repartidor), una **API REST** implementada tres veces con el mismo contrato (PHP, Python/Flask y Java/Spring Boot), un **frontend SPA en JavaScript** que detecta el backend disponible y una base de datos **MySQL** con cuatro tablas.

Capacidad	Estado	Evidencia en el código
Registro con verificación de mayoría de edad (≥ 18)	Implementado	registro_procesar.php; POST /register en los tres backends.
Login, sesión y cierre de sesión	Implementado	login_procesar.php, logout.php; POST /login.
Contraseñas con hash y sal (PBKDF2-SHA256, 120 000 iteraciones)	Implementado	config/config.php (hash_password), config.php de la API; formato común entre PHP, Java y Python.
Roles y control de acceso por rol	Implementado	require_role() en vistas; requireRole() en la API.
Gestión de usuarios y roles por el administrador	Implementado	admin/usuarios.php, admin/usuario.php; /clientes y /usuarios.
Gestión de productos y stock	Parcial	admin/producto*.php; sin categoría, foto, stock mínimo ni movimientos.

Capacidad	Estado	Evidencia en el código
Carrito, checkout con dirección y pedido de delivery	Implementado	cliente/carrito.php, checkout.php (el carrito vive en la sesión PHP).
POS: venta de mostrador con cobro inmediato	Implementado	cajero/index.php, venta.php, ventas.php.
Descuento de stock transaccional (bloqueo FOR UPDATE)	Implementado	checkout.php, venta.php, POST /ventas.
Entrega: entregado / no entregado (repone stock y anula el pago)	Implementado	repartidor/entrega.php; PUT /entregas/{id}.
Panel del administrador con totales y productos con stock bajo	Implementado	admin/index.php (umbral fijo de 5 unidades).
Pago por QR (SAS), caja, comprobantes	Pendiente	No existe código; solo el cobro en efectivo / contra entrega.
Asignación de repartidor, estado «en camino», notificaciones	Pendiente	El estado en_camino existe en la BD pero ningún flujo lo usa.
Promociones, categorías, fotos, WhatsApp, auditoría, respaldos	Pendiente	Sin tablas ni código.

3. Requisitos

3.1 Enfoque

Siguiendo *Ingeniería de Requerimientos* (Durán Quiroga), el documento distingue la **definición de requerimientos** (lo que el cliente necesita, en lenguaje natural), la **especificación de requerimientos** (detalle estructurado en casos de uso) y la **especificación de software** (diseño para los desarrolladores). Los requisitos se clasifican en funcionales (servicios que debe ofrecer el sistema) y no funcionales (restricciones de calidad y de desarrollo), y se verifican con cuatro controles: **validez**, **consistencia**, **completitud** y **realismo**. Cada requisito tiene un identificador para su trazabilidad hacia historias de usuario, casos de uso y componentes.

3.2 Requisitos funcionales

Los RF-01 a RF-22 provienen de la Visión del Proyecto. Se agregan cuatro requisitos derivados (RF-23 a RF-26) que la arquitectura necesita para cumplir los requisitos no funcionales. Prioridad: **A** alta, **M** media, **B** baja. Estado según el código actual.

ID	Requisito	Módulo	Prior.	Estado
RF-01	Mostrar un catálogo web con nombre, precio, foto, categoría y disponibilidad.	E-commerce	A	Parcial: sin foto ni categoría; exige sesión.
RF-02	Permitir al cliente armar un carrito con productos del catálogo.	E-commerce	A	Implementado (sesión PHP).
RF-03	Registrar ventas presenciales en el POS con total automático.	E-commerce	A	Implementado.
RF-04	Registrar el pago por QR (SAS, Banco Económico) o en efectivo.	Pagos	A	Parcial: solo efectivo / contra entrega.
RF-05	Registrar, actualizar y consultar stock desde la tablet de inventario.	E-commerce	A	Parcial: CRUD de productos; sin rol de bodega.
RF-06	Descontar automáticamente el stock al confirmar una venta o pedido.	E-commerce	A	Implementado.
RF-07	Generar alertas cuando el stock baje de un mínimo definido.	E-commerce	M	Parcial: umbral fijo ≤ 5 en el panel.
RF-08	Recibir y registrar pedidos hechos por WhatsApp y por la web.	E-commerce	M	Parcial: solo web.
RF-09	Permitir al cliente consultar el estado y seguimiento de su pedido.	Distribución	A	Parcial: ve el estado; sin «en preparación».
RF-10	Asignar un pedido a un repartidor y notificarle en su celular.	Distribución	A	Pendiente.
RF-11	Permitir al repartidor actualizar el estado (en camino, entregado…).	Distribución	A	Parcial: entregado / no entregado.
RF-12	Abrir caja, registrar movimientos y cerrar caja.	Pagos	M	Pendiente.
RF-13	Generar un comprobante o factura por cada venta, presencial u online.	Pagos	M	Pendiente.
RF-14	Crear y aplicar promociones o descuentos (productos, categorías, frecuentes).	E-commerce	M	Pendiente.

ID	Requisito	Módulo	Prior.	Estado
RF-15	Registrar clientes y consultar su historial de compras y pedidos.	E-commerce	M	Parcial.
RF-16	Validar la mayoría de edad antes de confirmar una venta o pedido.	Seguridad	A	Parcial: solo en el registro.
RF-17	Permitir a cualquier usuario crear cuenta y contraseña.	Seguridad	A	Implementado.
RF-18	Iniciar sesión y ver preferencias, descuentos, historial y promociones.	Seguridad	A	Parcial: sin descuentos ni preferencias.
RF-19	Permitir al administrador crear, editar, habilitar/deshabilitar y eliminar cuentas.	Seguridad	A	Implementado.
RF-20	Permitir al administrador asignar y modificar roles y permisos.	Seguridad	A	Implementado (roles).
RF-21	Supervisar de forma centralizada inventario, ventas, caja y pedidos.	E-commerce	A	Parcial: panel y compras; sin caja.
RF-22	Gestionar roles de acceso diferenciados (5 roles).	Seguridad	A	Implementado.
RF-23	*Derivado*. Registrar movimientos de inventario (entrada, ajuste, salida) con motivo y responsable.	E-commerce	A	Pendiente.
RF-24	*Derivado*. Gestionar categorías y fotos de producto, y el stock mínimo por producto.	E-commerce	M	Pendiente.
RF-25	*Derivado*. Registrar una bitácora de acciones administrativas sobre usuarios, inventario y pagos (RNF-05).	Seguridad	M	Pendiente.
RF-26	*Derivado*. Respaldar periódicamente la base de datos y permitir su restauración (RNF-06).	Seguridad	M	Pendiente.

3.3 Requisitos no funcionales

Cada requisito se acompaña de un criterio verificable y de la decisión arquitectónica que lo sostiene. Los valores marcados con **(*)** son metas propuestas por el equipo de arquitectura que deben validarse con datos reales de operación, como pide la propia Visión (RNF-07 y RNF-08).

ID	Categoría	Requisito	Criterio de verificación	Decisión que lo sostiene
RNF-01	Seguridad	Verificar la mayoría de edad antes de comprar bebidas alcohólicas.	Una compra de un usuario menor de 18 años se rechaza en el 100 % de los casos de prueba.	Regla en el servicio de pedidos, calculada desde la fecha de nacimiento.
RNF-02	Seguridad	Roles con permisos diferenciados.	Matriz RBAC (§7.14) probada por rol: 403 fuera de permiso.	Middleware de autorización por ruta y por propiedad del recurso.
RNF-03	Seguridad	Contraseñas cifradas con hash y sal.	Ninguna contraseña en texto plano en BD ni en logs.	PBKDF2-SHA256 con sal aleatoria; migrable a bcrypt/Argon2id.
RNF-04	Seguridad	Comunicación cifrada (HTTPS).	TLS 1.2 o superior; sin HTTP en producción.	Nginx con certificado y redirección 301.
RNF-05	Seguridad	Registro de acciones administrativas.	Cada alta, baja o cambio de usuario, rol, stock y pago deja un registro.	AuditoriaService suscrito a eventos del dominio.
RNF-06	Seguridad	Respaldos periódicos de la BD.	Respaldo diario cifrado y restauración probada. (*)	Tarea programada mysqldump a almacenamiento externo.

ID	Categoría	Requisito	Criterio de verificación	Decisión que lo sostiene
RNF-07	Rendimiento	Soportar el volumen diario sin degradar la respuesta.	Percentil 95 menor a 800 ms en operaciones del POS y del catálogo. (*)	Consultas indexadas, API sin estado, paginación.
RNF-08	Disponibilidad	Disponible durante el horario de atención.	Disponibilidad ≥ 99 % en horario de atención. (*) Modo sin conexión del POS: fase posterior.	Despliegue simple y reinicio automático; BD administrada.
RNF-09	Usabilidad	Interfaz simple para distintos niveles técnicos.	Una venta de mostrador se registra en menos de 6 acciones.	Flujos por rol con pantallas mínimas.
RNF-10	Portabilidad	Frontend responsivo para PC, tablet y celular.	Sin desplazamiento horizontal desde 360 px de ancho.	SPA con CSS responsive (mobile-first).
RNF-11	Mantenibilidad	Buenas prácticas de arquitectura.	Dependencias solo hacia abajo; sin acceso cruzado a tablas de otros módulos.	Arquitectura modular en capas (§7).
RNF-12	Confiabilidad	Consistencia e integridad ante fallos.	Ventas y stock se confirman en una sola transacción (todo o nada).	Transacciones ACID y bloqueo FOR UPDATE.

3.4 Restricciones y diferencias entre la Visión y el código

- **Backend:** la Visión fija Python; el repositorio contiene además PHP y Java. Ver ADR-02 (§7.16).
- **Frontend:** JavaScript con Node.js. Hoy es JavaScript sin proceso de construcción; Node.js se usará para herramientas (empaquetado, pruebas).
- **Base de datos:** MySQL. En desarrollo se usa XAMPP (MariaDB/MySQL local); en producción, MySQL 8 administrado.
- **Hash de contraseñas:** la Visión menciona bcrypt como ejemplo; el código usa PBKDF2-SHA256, que es un estándar aprobado y tiene el mismo formato en los tres backends. Se mantiene y se registra como decisión (ADR-05).
- **Pagos:** integración con el servicio SAS del Banco Económico. Su contrato técnico no es público; el diseño lo encapsula en un adaptador (§7.8) para no depender de detalles aún desconocidos.
- **Despliegue:** XAMPP en desarrollo; AWS u otro proveedor equivalente en producción.

> **Nota**
> Las diferencias anteriores no son errores: son decisiones que conviene dejar escritas. Cada una tiene su registro en §7.16.

3.5 Trazabilidad requisito → módulo

Módulo	Requisitos funcionales	Requisitos no funcionales relacionados
Seguridad	RF-16, RF-17, RF-18, RF-19, RF-20, RF-22, RF-25, RF-26	RNF-01 a RNF-06, RNF-12
E-commerce	RF-01, RF-02, RF-03, RF-05, RF-06, RF-07, RF-08, RF-14, RF-15, RF-21, RF-23, RF-24	RNF-07, RNF-09, RNF-10, RNF-12
Pagos	RF-04, RF-12, RF-13	RNF-04, RNF-05, RNF-12
Distribución	RF-09, RF-10, RF-11	RNF-08, RNF-10

4. Historias de usuario

Las historias describen cada requisito desde el punto de vista del usuario con el formato **«Como <rol>, quiero <acción>, para <beneficio>»**. Los criterios de aceptación usan la forma «Dado / Cuando / Entonces» abreviada, de modo que cada uno pueda convertirse en una prueba automatizada. Las historias marcadas como *Propuesta* aún no existen en el código.

Elemento	Convención
Identificador	HU-<módulo><número>: S (Seguridad), E (E-commerce), P (Pagos), D (Distribución).
Prioridad	A (alta, necesaria para operar), M (media), B (baja). Método MoSCoW simplificado.
Puntos de historia	Estimación relativa con la serie 1, 2, 3, 5, 8, 13. Una historia de 13 puntos debe dividirse.
Listo para desarrollar (DoR)	Tiene rol, valor, criterios de aceptación y dependencias identificadas.
Terminada (DoD)	Código revisado, pruebas automáticas pasando, endpoint documentado en OpenAPI, sin regresiones y validada por el dueño del producto.

4.1 Módulo de Seguridad

HU-S01 **Registrarme como cliente** — Prioridad: **A** · Puntos: **3**

Como visitante, **quiero** crear una cuenta con mis datos y una contraseña, **para** poder comprar y seguir mis pedidos.

Criterios de aceptación

- Dado que tengo 18 años o más, cuando completo nombre, apellido, CI, fecha de nacimiento, usuario, contraseña de al menos 8 caracteres y acepto los términos, entonces se crea la cuenta con rol *cliente* y la contraseña queda cifrada.
- Dado que soy menor de 18 años, cuando envío el formulario, entonces el sistema lo rechaza con un mensaje claro y no crea la cuenta.
- Dado que el usuario o el CI ya existen, entonces se responde con conflicto (409) y no se duplica el registro.

Requisitos: RF-16, RF-17 · Casos de uso: CU-S01 · Estado: Implementado

HU-S02 **Iniciar sesión** — Prioridad: **A** · Puntos: **3**

Como usuario del sistema, **quiero** ingresar con mi usuario y contraseña, **para** acceder a las funciones de mi rol.

Criterios de aceptación

- Dado que mis credenciales son válidas y mi cuenta está activa, entonces recibo acceso y veo el panel que corresponde a mi rol.
- Dado que las credenciales son inválidas o la cuenta está deshabilitada, entonces veo el mismo mensaje genérico («Usuario o contraseña incorrectos») sin revelar cuál dato falló.
- Después de 5 intentos fallidos seguidos, la cuenta se bloquea temporalmente (propuesta).

Requisitos: RF-18, RF-22 · Casos de uso: CU-S02 · Estado: Implementado (sin bloqueo)

HU-S03 **Cerrar sesión** — Prioridad: **M** · Puntos: **1**

Como usuario autenticado, **quiero** cerrar mi sesión, **para** evitar que otra persona use mi cuenta en un equipo compartido.

Criterios de aceptación

- Cuando cierro sesión, el token o la sesión deja de ser válido y se me redirige a la pantalla de ingreso.
- Una petición posterior con el token anterior responde 401.

Requisitos: RF-18 · Casos de uso: CU-S03 · Estado: Implementado

HU-S04	Administrar usuarios	Prioridad: **A** · Puntos: **5**

Como administrador, **quiero** crear, editar, habilitar, deshabilitar y eliminar cuentas, **para** controlar quién puede entrar al sistema.

Criterios de aceptación
- Solo el rol administrador accede a la administración de usuarios; cualquier otro rol recibe 403.
- No puedo eliminar mi propia cuenta en sesión.
- Si el usuario tiene ventas asociadas, el sistema lo **desactiva** en lugar de eliminarlo, para conservar el historial.

Requisitos: RF-19 · Casos de uso: CU-S04 · Estado: Implementado

HU-S05	Asignar roles y permisos	Prioridad: **A** · Puntos: **3**

Como administrador, **quiero** asignar o cambiar el rol de cada usuario (cliente, cajero, repartidor, bodega, administrador), **para** que cada persona vea solo lo que necesita.

Criterios de aceptación
- Al guardar un usuario puedo elegir su rol y el cargo.
- El cambio de rol se aplica en la siguiente petición del usuario afectado.
- Un cliente no puede quedar registrado con menos de 18 años aunque lo cree el administrador.

Requisitos: RF-20, RF-22 · Casos de uso: CU-S05 · Estado: Implementado

HU-S06	Consultar la bitácora de auditoría	Prioridad: **M** · Puntos: **5**

Como administrador, **quiero** ver quién hizo qué sobre usuarios, inventario y pagos, **para** investigar errores y detectar abusos.

Criterios de aceptación
- Cada alta, cambio o baja de usuario, rol, producto, stock y anulación de pago genera un registro con usuario, fecha, entidad y detalle.
- Puedo filtrar la bitácora por fecha, usuario y tipo de acción.

Requisitos: RF-25 · Casos de uso: CU-S06 · Estado: Propuesta

HU-S07	Respaldar y restaurar la base de datos	Prioridad: **M** · Puntos: **5**

Como administrador, **quiero** contar con copias de seguridad periódicas, **para** recuperar la información ante un fallo.

Criterios de aceptación
- Se genera un respaldo diario cifrado fuera del servidor de aplicación.
- Se documenta y prueba el procedimiento de restauración al menos una vez por mes.

Requisitos: RF-26 · Casos de uso: CU-S07 · Estado: Propuesta

4.2 Módulo de E-commerce

HU-E01	Ver el catálogo	Prioridad: **A** · Puntos: **3**

Como cliente o visitante, **quiero** ver los productos con nombre, foto, categoría, precio y disponibilidad, **para** elegir qué comprar.

Criterios de aceptación
- Solo se muestran productos activos y con stock mayor a cero.
- Puedo filtrar por categoría y buscar por nombre.
- El catálogo es público; para comprar debo iniciar sesión.

Requisitos: RF-01, RF-24 · Casos de uso: CU-E01 · Estado: Parcial

HU-E02	Armar mi carrito	Prioridad: **A** · Puntos: **3**

Como cliente, **quiero** agregar, quitar y vaciar productos de mi carrito, **para** revisar el total antes de confirmar.

Criterios de aceptación
- No puedo agregar más unidades que el stock disponible.
- El total se recalcula con el precio vigente al mostrar el carrito.
- Puedo vaciar el carrito con una confirmación.

Requisitos: RF-02 · Casos de uso: CU-E02 · Estado: Implementado

HU-E03	Confirmar mi pedido con entrega	Prioridad: **A** · Puntos: **8**

Como cliente, **quiero** registrar mi dirección y teléfono y confirmar el pedido, **para** recibir los productos en mi domicilio y pagar al recibirlos.

Criterios de aceptación
- La dirección y el teléfono son obligatorios.
- Si soy menor de 18 años, el pedido se rechaza.
- El stock se descuenta en la misma transacción que crea el pedido; si falta stock de cualquier producto no se crea nada y veo cuál faltó.
- El pedido queda en estado *pendiente* y el pago en *pendiente* hasta la entrega.

Requisitos: RF-02, RF-06, RF-16 · Casos de uso: CU-E03 · Estado: Implementado (sin validar edad)

HU-E04	Ver mis pedidos	Prioridad: **A** · Puntos: **2**

Como cliente, **quiero** consultar mi historial de pedidos con su estado, **para** saber en qué etapa está cada uno.

Criterios de aceptación
- Solo veo mis propios pedidos.
- Veo productos, total, dirección, estado del pedido, estado del pago, repartidor y observaciones.

Requisitos: RF-09, RF-15, RF-18 · Casos de uso: CU-E04 · Estado: Implementado

HU-E05	Registrar una venta en mostrador	Prioridad: **A** · Puntos: **5**

Como cajero, **quiero** seleccionar productos y cantidades y cobrar al instante, **para** atender rápido y sin errores de cálculo.

Criterios de aceptación
- El total se calcula automáticamente; puedo asociar un cliente registrado, pero es opcional.
- No puedo vender más que el stock disponible.
- La venta queda *pagada* y el stock se descuenta en una sola transacción.
- Veo el total cobrado y la cantidad de ventas del día.

Requisitos: RF-03, RF-06 · Casos de uso: CU-E05 · Estado: Implementado

HU-E06	Gestionar productos	Prioridad: **A** · Puntos: **5**

Como administrador, **quiero** crear, editar y desactivar productos con precio, stock, categoría y foto, **para** mantener el catálogo al día.

Criterios de aceptación
- El precio no puede ser negativo y el stock no puede ser menor que cero.
- Un producto con ventas se desactiva, no se borra.
- Solo administrador y bodega pueden modificar productos (el cajero solo los consulta).

Requisitos: RF-05, RF-24 · Casos de uso: CU-E06 · Estado: Parcial

HU-E07	Registrar entradas y ajustes de stock	Prioridad: A · Puntos: 5

Como encargado de bodega, **quiero** registrar desde la tablet las entradas de mercadería y los ajustes por conteo, **para** conocer el stock real en todo momento.

Criterios de aceptación
- Cada movimiento guarda producto, cantidad, motivo, usuario y fecha.
- El stock del producto se actualiza con el movimiento y nunca queda negativo.
- La pantalla es usable en tablet sin desplazamiento horizontal.

Requisitos: RF-05, RF-23 · Casos de uso: CU-E07 · Estado: Propuesta

HU-E08	Recibir alertas de stock bajo	Prioridad: M · Puntos: 3

Como administrador o encargado de bodega, **quiero** ser avisado cuando un producto cae por debajo de su stock mínimo, **para** reponer antes de quedarme sin existencias.

Criterios de aceptación
- Cada producto tiene su propio stock mínimo.
- El panel lista los productos bajo el mínimo, ordenados por urgencia.

Requisitos: RF-07, RF-24 · Casos de uso: CU-E08 · Estado: Parcial

HU-E09	Definir promociones y descuentos	Prioridad: M · Puntos: 8

Como administrador, **quiero** crear descuentos por producto, categoría o cliente frecuente con fechas de vigencia, **para** fidelizar clientes y mover inventario.

Criterios de aceptación
- Una promoción vencida no se aplica.
- Si hay varias promociones aplicables se usa la de mayor beneficio para el cliente.
- El comprobante muestra el descuento aplicado.

Requisitos: RF-14, RF-18 · Casos de uso: CU-E09 · Estado: Propuesta

HU-E10	Ver el panel de ventas	Prioridad: A · Puntos: 3

Como administrador, **quiero** ver totales de ventas, pedidos pendientes, monto cobrado y últimas ventas, **para** conocer el estado del negocio.

Criterios de aceptación
- El panel muestra productos y usuarios activos, ventas, pendientes y monto cobrado.
- Puedo abrir el detalle de cada compra.

Requisitos: RF-21 · Casos de uso: CU-E10 · Estado: Implementado

HU-E11	Registrar un pedido recibido por WhatsApp	Prioridad: M · Puntos: 5

Como cajero, **quiero** cargar en el sistema un pedido que llegó por WhatsApp, **para** mantener un solo flujo de pedidos y stock.

Criterios de aceptación
- El pedido se asocia a un cliente existente o a uno nuevo.
- Descuenta stock y entra al mismo flujo de entrega que los pedidos web.

Requisitos: RF-08 · Casos de uso: CU-E11 · Estado: Propuesta

4.3 Módulo de Pagos

HU-P01 **Cobrar en efectivo en el mostrador** Prioridad: **A** · Puntos: **2**

Como cajero, **quiero** registrar el cobro en efectivo de una venta, **para** dejar la venta pagada y el dinero registrado.

Criterios de aceptación
- El monto cobrado es igual al total de la venta.
- La venta queda con estado de pago *pagado* y fecha.

Requisitos: RF-04 · Casos de uso: CU-P01 · Estado: Implementado

HU-P02 **Pagar con código QR** Prioridad: **A** · Puntos: **13**

Como cliente o cajero, **quiero** pagar con un QR del Banco Económico, **para** no manejar efectivo.

Criterios de aceptación
- El sistema genera un QR dinámico con el monto exacto y un tiempo de vigencia.
- La venta solo se confirma cuando el banco notifica el pago y el monto y la referencia coinciden.
- Si el QR vence sin pago, la venta se anula y el stock se libera.
- La notificación del banco se valida (firma) y es idempotente: repetirla no duplica el cobro.

Requisitos: RF-04 · Casos de uso: CU-P02 · Estado: Propuesta (dividir en 2 historias)

HU-P03 **Registrar el cobro contra entrega** Prioridad: **A** · Puntos: **3**

Como repartidor, **quiero** confirmar que cobré al entregar el pedido, **para** que el pago del pedido quede registrado.

Criterios de aceptación
- Al confirmar la entrega, el monto cobrado pasa a ser igual al total y el pago queda *pagado*.
- Si no se entrega, no se registra ningún monto.

Requisitos: RF-04, RF-11 · Casos de uso: CU-P03 · Estado: Implementado

HU-P04 **Emitir comprobante de venta** Prioridad: **M** · Puntos: **5**

Como cajero, **quiero** imprimir o enviar un comprobante por cada venta, **para** respaldar la transacción ante el cliente.

Criterios de aceptación
- Cada venta pagada genera un comprobante con número correlativo único.
- Incluye detalle, descuentos, total, método de pago y fecha.

Requisitos: RF-13 · Casos de uso: CU-P04 · Estado: Propuesta

HU-P05 **Abrir y cerrar caja** Prioridad: **M** · Puntos: **8**

Como cajero, **quiero** abrir mi caja con un monto inicial y cerrarla con arqueo, **para** cuadrar el dinero del día.

Criterios de aceptación
- No puedo cobrar sin una caja abierta.
- Al cerrar, el sistema compara el efectivo esperado con el contado y registra la diferencia.
- Los ingresos y egresos manuales quedan registrados con concepto.

Requisitos: RF-12 · Casos de uso: CU-P05, CU-P06 · Estado: Propuesta

HU-P06 **Anular un pago** Prioridad: **M** · Puntos: **3**

Como administrador, **quiero** anular un pago con motivo, **para** corregir errores sin borrar historial.

Criterios de aceptación
- La anulación exige motivo y queda en la bitácora.
- Si el pedido no se entregó, el pago pasa automáticamente a *anulado* con monto cobrado cero.

Requisitos: RF-04, RF-25 · Casos de uso: CU-P07 · Estado: Parcial

4.4 Módulo de Distribución

HU-D01 **Ver mis entregas pendientes** Prioridad: **A** · Puntos: **3**

Como repartidor, **quiero** ver los pedidos por entregar con dirección, referencia, teléfono y monto, **para** planificar mi recorrido.

Criterios de aceptación
- Solo veo pedidos de tipo delivery en estado pendiente o en camino.
- Veo el contador de entregas realizadas y el monto cobrado.

Requisitos: RF-10, RF-11 · Casos de uso: CU-D01 · Estado: Implementado

HU-D02 **Asignar un pedido a un repartidor** Prioridad: **A** · Puntos: **5**

Como administrador, **quiero** asignar cada pedido a un repartidor específico, **para** evitar que dos repartidores vayan al mismo pedido.

Criterios de aceptación
- El repartidor asignado recibe el pedido en su lista y una notificación.
- Un repartidor solo ve los pedidos asignados a él.
- Puedo reasignar un pedido mientras no esté entregado.

Requisitos: RF-10 · Casos de uso: CU-D02 · Estado: Propuesta

HU-D03 **Marcar el pedido en camino** Prioridad: **M** · Puntos: **2**

Como repartidor, **quiero** avisar que salí a entregar, **para** que el cliente sepa que su pedido viene en camino.

Criterios de aceptación
- El estado cambia de *pendiente* a *en camino* y el cliente lo ve.
- No puedo marcar en camino un pedido de otro repartidor.

Requisitos: RF-09, RF-11 · Casos de uso: CU-D03 · Estado: Propuesta

HU-D04 **Confirmar la entrega** Prioridad: **A** · Puntos: **3**

Como repartidor, **quiero** confirmar que entregué el pedido, **para** cerrar el pedido y registrar el cobro.

Criterios de aceptación
- El pedido pasa a *entregada*, el pago a *pagado* y se guarda mi identificador y la fecha.
- Si otro repartidor ya procesó el pedido, veo «El pedido ya fue procesado» y no se duplica el cobro.

Requisitos: RF-11 · Casos de uso: CU-D04 · Estado: Implementado

HU-D05 **Marcar como no entregado** Prioridad: **A** · Puntos: **5**

Como repartidor, **quiero** registrar que no pude entregar y el motivo, **para** devolver los productos al inventario.

Criterios de aceptación
- El pedido pasa a *no entregado* y el pago a *anulado* con monto cero.
- Las cantidades vuelven al stock en la misma transacción.
- La observación queda visible para el cliente y el administrador.

Requisitos: RF-06, RF-11 · Casos de uso: CU-D05 · Estado: Implementado

HU-D06	Consultar mi historial de entregas	Prioridad: **B** · Puntos: **2**

Como repartidor, **quiero** ver mis entregas anteriores, **para** saber cuánto cobré y cuántos pedidos atendí.

Criterios de aceptación

- Veo cliente, dirección, resultado, monto cobrado, fecha y observación de cada pedido atendido.

Requisitos: RF-11 · Casos de uso: CU-D06 · Estado: Implementado

HU-D07	Seguir el estado de mi pedido	Prioridad: **A** · Puntos: **5**

Como cliente, **quiero** ver en qué estado está mi pedido y quién lo lleva, **para** saber cuándo llega.

Criterios de aceptación

- Veo los estados recibido, en preparación, en camino y entregado.
- Recibo una notificación al cambiar el estado (WhatsApp, propuesta).

Requisitos: RF-09 · Casos de uso: CU-D07, CU-D09 · Estado: Parcial

5. Especificación de casos de uso

Los casos de uso describen el comportamiento del sistema desde el punto de vista del usuario y delimitan el sistema frente a su entorno (Jacobson; Letelier). Los **diagramas** de cada módulo están en la sección 8.1; aquí se incluyen los **actores** y la **especificación detallada** de los casos más importantes, con la plantilla del material de clase (versión, fuentes, precondición, secuencia normal, postcondición, excepciones, rendimiento, frecuencia, importancia y urgencia).

5.1 Actores

Actor	Tipo	Descripción
Visitante	Principal	Persona sin sesión iniciada que ve el catálogo o crea una cuenta.
Cliente	Principal	Usuario registrado que compra, paga y sigue sus pedidos.
Cajero	Principal	Registra ventas de mostrador, cobra y administra su caja.
Encargado de bodega	Principal	Registra entradas y ajustes de stock desde la tablet (rol *empleado* en el código).
Repartidor	Principal	Entrega pedidos y registra el resultado y el cobro.
Administrador	Secundario	Mantiene usuarios, roles, productos, promociones y supervisa el negocio.
Banco Económico (SAS)	Sistema externo	Genera códigos QR y notifica los pagos recibidos.
WhatsApp Business	Sistema externo	Canal de pedidos y de notificaciones al cliente (propuesto).
Planificador de tareas	Sistema	Dispara respaldos y revisiones de stock y de QR vencidos (propuesto).

La misma persona puede actuar con varios roles; el nombre del actor describe el papel que desempeña, no a la persona.

5.2 Especificaciones detalladas

Caso de uso	CU-S01 — Registrar cuenta
Versión	1.0 (31/10/2026)
Autores	Grupo N1 — J. L. Bayo, D. Beltrán, F. C. Castro
Fuentes	Visión del Proyecto LicoFast; código del repositorio (versión analizada).
Objetivos asociados	Permitir que un cliente cree su cuenta de forma autónoma y segura.
Requisitos / HU	RF-16, RF-17 · HU-S01
Actor principal	Visitante
Descripción	El sistema se comporta como se describe cuando el visitante envía el formulario de registro.
Precondición	El visitante no tiene sesión iniciada.
Secuencia normal	**Paso** — **Acción** 1 — El visitante ingresa nombre, apellido, CI, fecha de nacimiento, teléfono (opcional), usuario y contraseña, y acepta los términos. 2 — El sistema valida los campos obligatorios y que la contraseña tenga al menos 8 caracteres. 3 — Se realiza el caso de uso **Verificar mayoría de edad**: calcula la edad exacta desde la fecha de nacimiento. 4 — El sistema comprueba que el usuario y el CI no existan. 5 — Se realiza **Cifrar contraseña**: genera una sal aleatoria y guarda el hash. 6 — El sistema crea el usuario con rol *cliente*, activo, y confirma el registro.
Postcondición	Existe un usuario con rol cliente y contraseña cifrada; puede iniciar sesión.
Excepciones	**Paso** — **Acción** 1 — Si falta un dato o la contraseña es corta, el sistema informa el error y el caso de uso continúa en el paso 1. 2 — Si el visitante es menor de 18 años, el sistema rechaza el registro y el caso de uso aborta. 3 — Si el usuario o el CI ya existen, el sistema responde con conflicto (409) y el caso de uso continúa en el paso 1.
Rendimiento	Paso 5: menos de 500 ms (el hash es deliberadamente costoso). Total: menos de 2 s.
Frecuencia esperada	10 veces por semana (estimado)
Importancia	Importante
Urgencia	Hay presión
Comentarios	Código actual: registro_procesar.php y POST /register. El backend Python calcula la edad con días / 365; debe usar la fecha exacta (ver §7.17).

Caso de uso	CU-S02 — Iniciar sesión
Versión	1.0 (31/10/2026)
Autores	Grupo N1 — J. L. Bayo, D. Beltrán, F. C. Castro
Fuentes	Visión del Proyecto LicoFast; código del repositorio (versión analizada).
Objetivos asociados	Autenticar al usuario y entregarle solo las funciones de su rol.
Requisitos / HU	RF-18, RF-22 · HU-S02
Actor principal	Cliente, cajero, bodega, repartidor o administrador
Descripción	El sistema se comporta como se describe cuando un usuario envía sus credenciales.

Precondición	El usuario tiene una cuenta activa.
Secuencia normal	**Paso** / **Acción** 1 — El usuario ingresa su nombre de usuario y contraseña. 2 — El sistema busca la cuenta por nombre de usuario. 3 — Se realiza **Validar credenciales y emitir token**: compara la contraseña con el hash almacenado. 4 — El sistema emite un token firmado con identificador, rol y vencimiento. 5 — El sistema devuelve el usuario (sin su hash) y el token; el cliente muestra el panel del rol.
Postcondición	El usuario tiene una sesión válida con su rol.
Excepciones	**Paso** / **Acción** 1 — Si la cuenta no existe, está inactiva o la contraseña no coincide, el sistema responde 401 con un mensaje genérico y el caso de uso aborta. 2 — Si hay 5 intentos fallidos seguidos, se realiza **Bloquear cuenta** (propuesta) y el caso de uso aborta.
Rendimiento	Respuesta en menos de 1 s.
Frecuencia esperada	60 veces por día (estimado)
Importancia	Vital
Urgencia	Inmediatamente
Comentarios	Hoy el token es «id|rol» sin firma (§7.17, hallazgo H-01): debe reemplazarse por JWT.

Caso de uso	**CU-E03 — Confirmar pedido (checkout)**
Versión	1.0 (31/10/2026)
Autores	Grupo N1 — J. L. Bayo, D. Beltrán, F. C. Castro
Fuentes	Visión del Proyecto LicoFast; código del repositorio (versión analizada).
Objetivos asociados	Registrar un pedido de delivery con su dirección y reservar el stock.
Requisitos / HU	RF-02, RF-06, RF-16 · HU-E03
Actor principal	Cliente
Descripción	El sistema se comporta como se describe cuando el cliente confirma su carrito con los datos de entrega.
Precondición	El cliente inició sesión y su carrito tiene al menos un producto.
Secuencia normal	**Paso** / **Acción** 1 — El cliente revisa el resumen y registra dirección, referencia (opcional) y teléfono de contacto. 2 — El sistema valida que dirección y teléfono no estén vacíos. 3 — Se realiza **Verificar mayoría de edad** del cliente. 4 — El sistema inicia una transacción. 5 — Por cada producto se realiza **Verificar stock disponible** con bloqueo de fila y luego **Descontar stock**. 6 — Si hay una promoción vigente, se realiza **Aplicar promoción** (propuesta). 7 — El sistema crea la venta tipo delivery con estado y pago *pendiente* y su detalle. 8 — El sistema confirma la transacción, vacía el carrito y muestra el número de pedido.
Postcondición	Existe un pedido pendiente con stock reservado, visible para el cliente y para el repartidor.

Excepciones	**Paso** — **Acción** 1 — Si faltan datos de entrega, el sistema informa y el caso de uso continúa en el paso 1. 2 — Si el cliente es menor de edad, el sistema rechaza el pedido y el caso de uso aborta. 3 — Si algún producto no tiene stock suficiente, el sistema revierte la transacción, indica el producto y el caso de uso aborta.
Rendimiento	Pasos 4 a 8: menos de 1,5 s.
Frecuencia esperada	30 pedidos por día (estimado)
Importancia	Vital
Urgencia	Inmediatamente
Comentarios	Si el pedido nunca se atiende, el stock queda reservado: ver propuesta de caducidad en §7.17 (H-08).

Caso de uso	**CU-E05 — Registrar venta en mostrador (POS)**
Versión	1.0 (31/10/2026)
Autores	Grupo N1 — J. L. Bayo, D. Beltrán, F. C. Castro
Fuentes	Visión del Proyecto LicoFast; código del repositorio (versión analizada).
Objetivos asociados	Registrar y cobrar una venta presencial en pocos pasos.
Requisitos / HU	RF-03, RF-06 · HU-E05
Actor principal	Cajero
Descripción	El sistema se comporta como se describe cuando el cajero registra una compra en el mostrador.
Precondición	El cajero inició sesión (y, en la versión con caja, tiene una caja abierta).
Secuencia normal	**Paso** — **Acción** 1 — El cajero elige, de forma opcional, un cliente registrado. 2 — Ingresa la cantidad de cada producto vendido. 3 — El sistema inicia la transacción y, por cada producto, realiza **Verificar stock disponible** y **Descontar stock**. 4 — El sistema calcula el total. 5 — Se realiza **Cobrar en efectivo** (o **Cobrar con QR**): la venta queda pagada. 6 — El sistema confirma la transacción y muestra el número de compra y el total.
Postcondición	Existe una venta de mostrador pagada y el stock fue descontado.
Excepciones	**Paso** — **Acción** 1 — Si ninguna cantidad es mayor que cero, el sistema informa y el caso de uso continúa en el paso 2. 2 — Si falta stock, el sistema revierte y el caso de uso aborta.
Rendimiento	Menos de 1 s por venta de hasta 10 productos.
Frecuencia esperada	80 veces por día (estimado)
Importancia	Vital
Urgencia	Inmediatamente

Caso de uso	**CU-P02 — Cobrar con código QR**
Versión	1.0 (31/10/2026)
Autores	Grupo N1 — J. L. Bayo, D. Beltrán, F. C. Castro

Fuentes	Visión del Proyecto LicoFast; código del repositorio (versión analizada).
Objetivos asociados	Cobrar una venta por QR sin manejar efectivo y sin riesgo de doble cobro.
Requisitos / HU	RF-04 · HU-P02
Actor principal	Cajero o cliente; Banco Económico (SAS)
Descripción	El sistema se comporta como se describe cuando se elige el pago por QR para una venta pendiente.
Precondición	Existe una venta con stock reservado y estado de pago pendiente.
Secuencia normal	**Paso** — **Acción** 1 — El actor elige el método QR. 2 — Se realiza **Generar QR dinámico**: el sistema pide al banco un QR con monto exacto y referencia única. 3 — El sistema muestra el QR y espera la confirmación. 4 — El cliente paga desde la aplicación de su banco. 5 — El banco notifica el pago al sistema (webhook). 6 — Se realiza **Confirmar pago recibido**: valida firma, monto y referencia. 7 — El sistema marca la venta como pagada y emite el comprobante.
Postcondición	La venta está pagada y tiene comprobante; el cobro quedó registrado una sola vez.
Excepciones	**Paso** — **Acción** 1 — Si el QR vence sin pago, el sistema marca el pago como vencido. En una venta de mostrador anula la venta y libera el stock, y el caso de uso aborta; en un pedido de delivery el pedido sigue pendiente y puede pagarse con un QR nuevo o contra entrega. 2 — Si el monto o la referencia no coinciden, el sistema ignora la notificación, registra la alerta y el caso de uso continúa en el paso 3. 3 — Si la notificación llega repetida, el sistema la ignora (idempotencia). 4 — Si el banco no responde al generar el QR, el sistema ofrece cobrar en efectivo.
Rendimiento	Paso 2: menos de 3 s. Paso 6: menos de 1 s.
Frecuencia esperada	20 veces por día (estimado)
Importancia	Importante
Urgencia	Puede esperar
Comentarios	Depende del contrato técnico del servicio SAS, aún por obtener del banco. Se encapsula en el adaptador QrSasAdapter (§7.8).

Caso de uso	**CU-D04 / CU-D05 — Confirmar entrega o marcar como no entregado**
Versión	1.0 (31/10/2026)
Autores	Grupo N1 — J. L. Bayo, D. Beltrán, F. C. Castro
Fuentes	Visión del Proyecto LicoFast; código del repositorio (versión analizada).
Objetivos asociados	Cerrar el pedido con su resultado real, conservando la integridad del stock y del cobro.
Requisitos / HU	RF-06, RF-11 · HU-D04, HU-D05
Actor principal	Repartidor
Descripción	El sistema se comporta como se describe cuando el repartidor registra el resultado de un pedido.
Precondición	El repartidor inició sesión y el pedido está *pendiente* o *en camino*.

Secuencia normal	**Paso** / **Acción** 1 — El repartidor abre el pedido y revisa cliente, dirección, teléfono y monto a cobrar. 2 — El repartidor elige **Entregado** o **No entregado** y escribe una observación opcional. 3 — El sistema inicia una transacción y bloquea el pedido. 4 — Si eligió Entregado: se realiza **Registrar cobro contra entrega**; el pedido pasa a *entregada*, el pago a *pagado* y el monto cobrado es igual al total. 5 — Si eligió No entregado: se realiza **Reponer stock**; el pedido pasa a *no entregado*, el pago a *anulado* y el monto cobrado es cero. 6 — El sistema confirma la transacción y (propuesta) notifica al cliente.
Postcondición	El pedido queda cerrado, con repartidor, fecha y observación; el stock y el cobro son coherentes.
Excepciones	**Paso** / **Acción** 1 — Si otro repartidor ya procesó el pedido, el sistema informa «El pedido ya fue procesado» y el caso de uso aborta sin cambios. 2 — Si no se elige ninguna opción, el sistema lo indica y el caso de uso continúa en el paso 2.
Rendimiento	Menos de 1 s.
Frecuencia esperada	30 veces por día (estimado)
Importancia	Vital
Urgencia	Inmediatamente
Comentarios	Código actual: repartidor/entrega.php y PUT /entregas/{id}. Es el ejemplo de transacción más completo del proyecto.

6. Pila de desarrollo (stack tecnológico)

La pila se eligió por tres criterios: que el equipo ya la domina, que tenga costo bajo para una licorería pequeña y que permita crecer sin reescribir. La tabla distingue la **tecnología objetivo** de lo que existe hoy en el repositorio, porque el proyecto mantiene prototipos en tres lenguajes.

Capa	Tecnología objetivo	Estado en el repositorio	Justificación
Cliente web	HTML5, CSS3 y JavaScript (SPA, *fetch* a la API). Node.js solo para herramientas de desarrollo.	Presente: `frontend/` (index.html, app.js, config.js, style.css).	Sin dependencias de ejecución; carga rápida en celular y en la tablet del bodeguero.
Servicio REST	**Python 3 + Flask 3.1.2**, flask-cors 6.0.1, Gunicorn como servidor de aplicación.	Presente: `backend-python/app.py` (requirements.txt fija las versiones).	Flask es ligero y expresa bien un servicio REST; el equipo ya lo usa. Ver ADR-02.
Acceso a datos	mysql-connector-python 9.4.0 con *pool* de conexiones y consultas parametrizadas.	Presente, pero abre una conexión por consulta.	Evita inyección SQL y soporta transacciones explícitas con `SELECT ... FOR UPDATE`.
Base de datos	**MySQL 8** (InnoDB, utf8mb4).	Presente: `database/licofast.sql` (4 tablas).	InnoDB aporta transacciones y claves foráneas; es el motor de XAMPP y de Amazon RDS.
Autenticación	JWT (HS256) y PBKDF2-SHA256 con sal por usuario.	Hash presente (120 000 iteraciones). El token es «id|rol» sin firma.	Estándar, sin estado en el servidor. Ver ADR-04 y ADR-05.
Entorno de desarrollo	XAMPP (Apache + MySQL + PHP 8).	Presente: `backend-php/` y las páginas PHP de `admin/` `cajero/` `cliente/` `repartidor/`.	Es el entorno con el que se construyó el prototipo; sirve como referencia funcional.
Alternativa evaluada	Spring Boot 3.5.5 (Java 17).	Esqueleto en `backend-java/`, sin lógica de negocio.	Descartada para producción: más código y más memoria sin beneficio para este volumen. Ver ADR-02.
Servidor web	Nginx como proxy inverso con TLS (Let's Encrypt).	Propuesto.	Termina HTTPS, sirve los archivos estáticos y limita la tasa de peticiones.
Infraestructura	AWS: EC2 (aplicación), RDS MySQL, S3 (respaldos y fotos).	Propuesto (sección 8.4).	Servicios administrados reducen la operación. Sustituible por un VPS.
Integraciones	API SAS del Banco Económico (QR); WhatsApp Business Cloud API.	Propuesto. Sin contrato técnico aún.	Son los dos canales que pide la Visión del Proyecto.
Calidad y entrega	pytest, Postman/Newman, Git y GitHub Actions; mysqldump con cron.	Git y GitHub presentes.	Pruebas automáticas de la API en cada cambio.

Reproducibilidad
Versiones fijadas en `requirements.txt`: Flask 3.1.2, flask-cors 6.0.1, mysql-connector-python 9.4.0. Fijarlas permite reproducir el entorno en el servidor de producción.

7. Arquitectura de software

Esta sección es el núcleo del documento. Sigue el orden que propone el curso: primero qué es una arquitectura y para qué sirve, luego los factores que la condicionan, los estilos candidatos y la decisión, y por último las vistas detalladas de cada módulo.

7.1 Fundamentos

Definición

Según la norma IEEE 1471, la **arquitectura de software** es la organización fundamental de un sistema, expresada en sus **componentes**, las **relaciones** entre ellos y con el entorno, y los **principios** que guían su diseño y su evolución. En términos prácticos: define las piezas grandes, cómo se hablan y por qué se hicieron así.

Razones para tener una arquitectura

Razón (material del curso)	Qué significa para LicoFast
Extensión	Agregar Finanzas completas, WhatsApp o una app móvil sin tocar lo que ya funciona. Cada módulo expone una fachada; los nuevos canales usan la misma API.
Cambio	El contrato del banco aún no existe. El pago por QR queda detrás de una interfaz (Strategy), de modo que cambiar de proveedor afecta un solo adaptador.
Simplicidad	Un equipo de tres personas necesita pocas piezas: un monolito modular con cuatro módulos y capas claras, no una red de servicios.
Eficiencia	Las operaciones críticas (descontar stock, cobrar) son transacciones locales cortas; en un solo proceso no pagan latencia de red entre servicios.

La arquitectura soporta la funcionalidad

La funcionalidad (los casos de uso) es lo que el usuario ve; la arquitectura es lo que permite que esa funcionalidad sea rápida, segura y modificable. Por eso este documento parte de los requisitos (sección 3), toma de ellos los **atributos de calidad** que más pesan y recién entonces elige la estructura.

Proceso de diseño seguido

1. Entender el problema y los requisitos (secciones 2 a 5).
2. Identificar los factores arquitectónicamente significativos (7.2).
3. Evaluar estilos y escoger uno con sus alternativas (7.3 y 7.4).
4. Descomponer en módulos con alta cohesión y bajo acoplamiento (7.5).
5. Detallar cada módulo, su API y sus datos (7.6 a 7.11).
6. Describir el comportamiento y la seguridad (7.12 a 7.14).
7. Verificar con escenarios de calidad y registrar las decisiones (7.15 y 7.16).

7.2 Factores que condicionan la arquitectura

Factor	Descripción	Efecto en el diseño
Dominio regulado	Venta de alcohol: solo mayores de 18 años y trazabilidad.	Verificación de edad en el servidor, no solo en pantalla; auditoría.
Integridad de stock	Una botella no puede venderse dos veces (mostrador y delivery compiten).	Transacciones con bloqueo de fila; reglas de stock en un solo lugar.
Cuatro tipos de usuario y tres dispositivos	Celular del cliente, PC del cajero, tablet del bodeguero y celular del repartidor.	API única, independiente del cliente; control de acceso por rol.
Equipo y plazo	Tres estudiantes, un semestre.	Monolito modular, pila conocida, pocas piezas operativas.
Pago externo incierto	Contrato SAS del banco pendiente.	Adaptador y Strategy para aislar el cambio.
Presupuesto	Negocio pequeño, sin personal de sistemas.	Servicios administrados y despliegue simple; respaldo automático.
Volumen bajo a medio	Decenas de pedidos y cientos de ventas por día.	Escala vertical suficiente; sin necesidad de microservicios.

Atributos de calidad priorizados

Prioridad	Atributo	Razón	Táctica principal
1	Seguridad e integridad	Dinero, datos personales y venta restringida.	JWT firmado, RBAC, transacciones, auditoría.
2	Disponibilidad en horario comercial	Una caja caída detiene la venta.	Reinicio automático, respaldo, *health check*.
3	Modificabilidad	El banco y el negocio cambiarán reglas.	Módulos, fachadas, Strategy, Repository.
4	Usabilidad	Cajero y repartidor operan con apuro.	Pocos pasos por tarea; pantallas por rol.
5	Rendimiento	Respuesta ágil en caja.	Índices, consultas cortas, *pool*.

7.3 Estilos arquitectónicos evaluados

El curso presenta cinco familias de modelos (Shaw y Garlan). Se evaluó cada una frente a los factores anteriores.

Familia	Idea	Aplicación en LicoFast	Decisión
Flujo de datos (*pipes and filters*)	Datos que pasan por filtros en secuencia.	Útil solo para reportes por lotes.	No como estilo base
Componentes independientes (cliente-servidor, eventos)	Componentes que se comunican por mensajes o peticiones.	**Cliente-servidor** entre el navegador y la API REST; **eventos** internos para auditoría y notificaciones.	**Adoptado** (externo)
Máquina virtual	Intérprete de reglas o lenguaje.	Sin necesidad.	No
Centrado en datos (repositorio)	Componentes alrededor de un almacén común.	Una sola base MySQL compartida con propiedad clara por módulo.	Adoptado (datos)
En capas	Cada capa usa solo la inferior.	Presentación, interfaz, lógica, acceso a datos y almacén (modelo de 5 capas del curso).	**Adoptado** (interno)

Alternativas de despliegue

Criterio	Monolito modular	Microservicios	Serverless (funciones)
Complejidad operativa	Baja	Alta (red, descubrimiento, trazas)	Media
Transacción stock + venta	Una transacción local	Requiere sagas o consistencia eventual	Difícil
Costo con volumen bajo	Bajo	Alto	Bajo
Equipo de 3 personas	Adecuado	Excesivo	Aceptable
Evolución futura	Se puede extraer un módulo si crece	Ya separado	Acoplado al proveedor
Resultado	**Elegido**	Descartado	Descartado

Un cambio de venta con descuento de stock debe ser atómico; en microservicios exigiría coordinar varios servicios para algo que en un monolito resuelve una sola transacción de base de datos. Ver ADR-01.

7.4 Estilo adoptado: servicio REST en capas, con módulos

LicoFast se construye como **un servicio web REST** (un solo proceso desplegable) organizado en **cuatro módulos** de negocio. Dentro de cada módulo se aplica el **modelo empresarial de cinco capas** del curso. Los clientes (web de cliente, panel de administrador, caja, tablet de bodega y celular del repartidor) consumen la misma API.

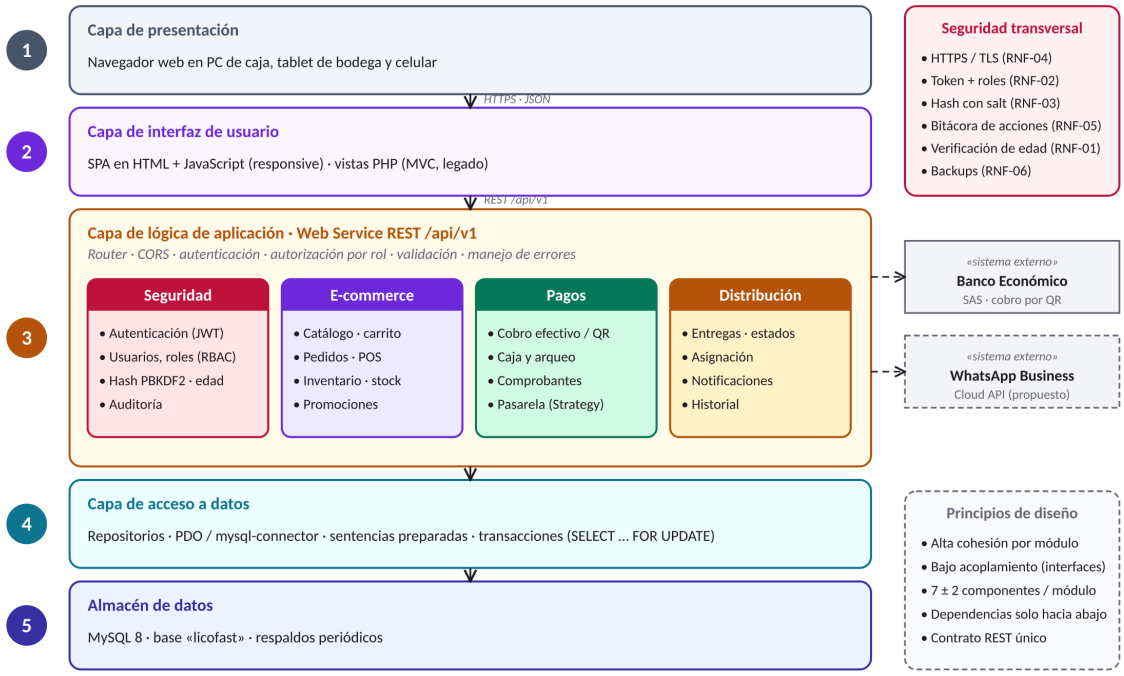


Figura 1. Arquitectura en cinco capas de LicoFast, con seguridad transversal y sistemas externos.

Responsabilidad de cada capa

Capa	Responsabilidad	Elementos en LicoFast	Regla de dependencia
1. Presentación	Mostrar y capturar.	Navegador, SPA, páginas PHP del prototipo.	Solo llama a la capa 2.
2. Interfaz de usuario / API	Recibir peticiones, validar forma, autenticar y autorizar, dar formato JSON.	Controladores REST, middleware JWT, validadores, manejo de errores (Front Controller).	Llama a la capa 3. No contiene reglas de negocio.
3. Lógica de aplicación	Reglas del negocio y transacciones.	Servicios por módulo (*Service Layer*), máquina de estados del pedido, estrategias de pago, fachadas.	Llama a la capa 4 y publica eventos.
4. Acceso a datos	Traducir entre objetos y SQL.	Repositorios (Repository), conexión y *pool*, consultas parametrizadas.	Solo llama a la capa 5.
5. Almacén de datos	Persistir.	MySQL 8 (InnoDB), respaldos en S3.	—

> **Del prototipo a la arquitectura**
> El prototipo mezcla las capas 2, 3 y 4 en un solo archivo (`app.py`, `index.php`). La primera iteración del plan (sección 9) los separa sin cambiar el comportamiento visible. La tabla del anexo C relaciona cada carpeta del repositorio con su capa objetivo.

Patrón MVC dentro del cliente y del servicio

El curso presenta MVC como separación entre modelo, vista y controlador. En LicoFast: la **vista** es la SPA (o las páginas PHP del prototipo), el **controlador** son los endpoints REST y el **modelo** son las entidades de dominio con sus servicios y repositorios. Como la vista está desacoplada por HTTP, puede cambiarse (por ejemplo, a una aplicación móvil) sin tocar el modelo.

Principios de diseño aplicados

- **Alta cohesión:** cada módulo reúne funciones que cambian por la misma razón (todo lo del pago, juntos).
- **Bajo acoplamiento:** los módulos se hablan solo mediante su fachada, nunca consultando tablas ajenas.
- **Agrupación 7 ± 2:** cada módulo tiene entre 5 y 9 componentes principales, para que una persona los abarque a la vez.
- **Una sola fuente de verdad** para cada regla (stock, edad, estado del pedido).
- **Falla segura:** ante la duda, la transacción se revierte.

7.5 Descomposición modular y dependencias

La división en cuatro módulos responde a los casos de uso agrupados en la sección 5 y a la pregunta «qué cambia junto».

Módulo	Responsabilidad	Es dueño de las tablas	Casos de uso
Seguridad	Identidad, acceso, roles, auditoría.	usuarios, auditoria	CU-S01 a S08
E-commerce	Catálogo, promociones, carrito, pedidos, mostrador e inventario.	productos, categorias, promociones, ventas, detalle_venta, movimientos_stock	CU-E01 a E11
Pagos	Métodos de pago, QR, caja, cobranza y comprobantes.	pagos, cajas, movimientos_caja, comprobantes	CU-P01 a P08
Distribución	Asignación y entrega de pedidos, resultado y cobro contra entrega.	entregas	CU-D01 a D09

Matriz de dependencias

Una celda indica que el módulo de la fila **usa** la fachada del módulo de la columna.

Usa →	Seguridad	E-commerce	Pagos	Distribución
Seguridad	—			
E-commerce	Identidad y rol	—	Solicita cobro	
Pagos	Identidad y rol	Marca venta pagada	—	
Distribución	Identidad y rol	Lee pedidos; repone stock	Registra cobro	—

Control del acoplamiento
Hay un ciclo aparente entre E-commerce y Pagos (E-commerce pide el cobro y Pagos marca la venta como pagada). Se rompe con **eventos**: Pagos publica *PagoConfirmado* y E-commerce lo escucha; E-commerce no importa clases de Pagos, solo su interfaz *PasarelaPago*. Distribución depende de E-commerce y Pagos, pero ninguno depende de Distribución, por lo que puede extraerse como servicio si algún día fuera necesario.

7.6 Módulo Seguridad

Gestiona quién es cada usuario, qué puede hacer y qué hizo. Es la puerta de entrada de los otros tres módulos: todos le piden la identidad y el rol de cada petición.

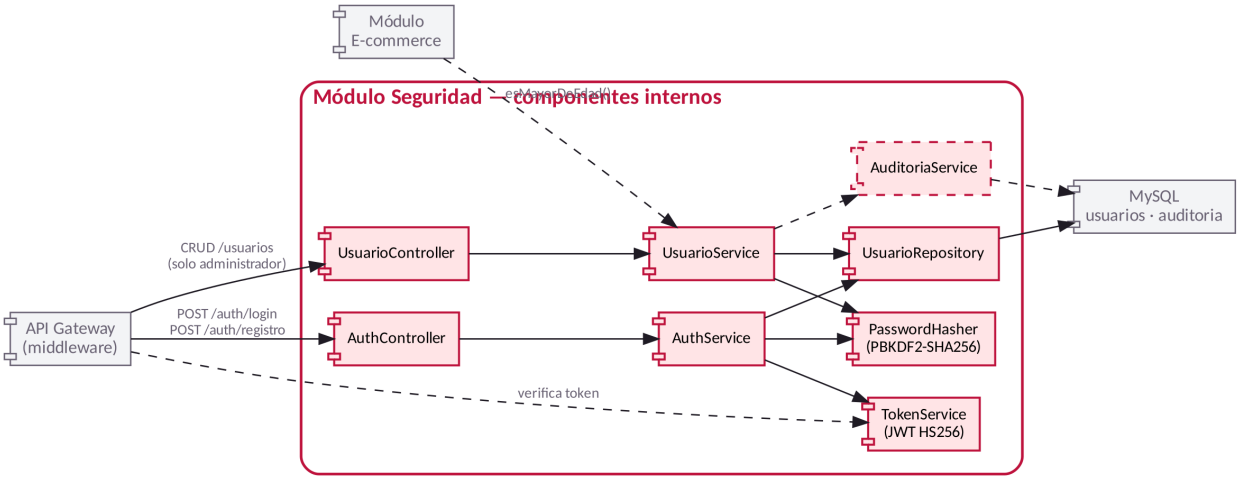


Figura 2. Componentes del módulo Seguridad.

Componentes

Componente	Capa	Responsabilidad
AuthController	Interfaz	Expone login, registro y perfil.
UsuarioController	Interfaz	CRUD de usuarios y roles (solo administrador).
JwtMiddleware	Interfaz	Valida el token y fija la identidad en el contexto de la petición.
RbacGuard	Interfaz	Decorador que exige rol y comprueba la propiedad del recurso.
AuthService	Lógica	Valida credenciales, emite tokens, bloquea cuentas.
UsuarioService	Lógica	Reglas de alta, edad mínima, baja lógica.
PasswordHasher	Lógica	PBKDF2-SHA256 con sal y comparación en tiempo constante.
AuditoriaService	Lógica	Registra quién hizo qué (escucha eventos).
UsuarioRepository	Datos	Acceso a la tabla usuarios.

Reglas de negocio

- Solo se registran personas de 18 años o más, calculado con la fecha exacta.
- Usuario y CI son únicos.
- Un usuario inactivo no puede iniciar sesión.
- El administrador no puede eliminarse a sí mismo; los usuarios con historial se desactivan, no se borran.
- Cinco intentos fallidos consecutivos bloquean la cuenta 15 minutos (propuesto).
- El mensaje de error de login es genérico.

Datos

Tablas `usuarios` (existente) y `auditoria` (propuesta). El campo `password` guarda `PBKDF2-SHA256$iteraciones$sal$hash`.

Patrones aplicados

- **Front Controller / Middleware** para autenticar y autorizar una sola vez.
- **Observer:** la auditoría escucha eventos de los demás módulos.
- **Facade** *SeguridadFacade*: `identidad(token)` y `tienePermiso(rol, recurso)`.

Endpoints principales

Método y ruta	Rol	Descripción
POST /api/v1/auth/login	Público	Valida credenciales y devuelve el JWT.
POST /api/v1/auth/register	Público	Crea una cuenta de cliente.
GET /api/v1/auth/me	Autenticado	Datos del usuario de la sesión.
GET /api/v1/usuarios	Administrador	Lista, con filtro por rol.
POST /api/v1/usuarios	Administrador	Crea personal con rol.
PUT /api/v1/usuarios/{id}	Administrador	Modifica datos, rol o estado.
DELETE /api/v1/usuarios/{id}	Administrador	Baja lógica (o física si no tiene historial).

7.7 Módulo E-commerce

Cubre todo lo que ocurre entre ver un producto y dejar la venta registrada: catálogo, promociones, carrito, pedido de delivery, venta de mostrador e inventario. Es el módulo con más reglas y el dueño del stock.

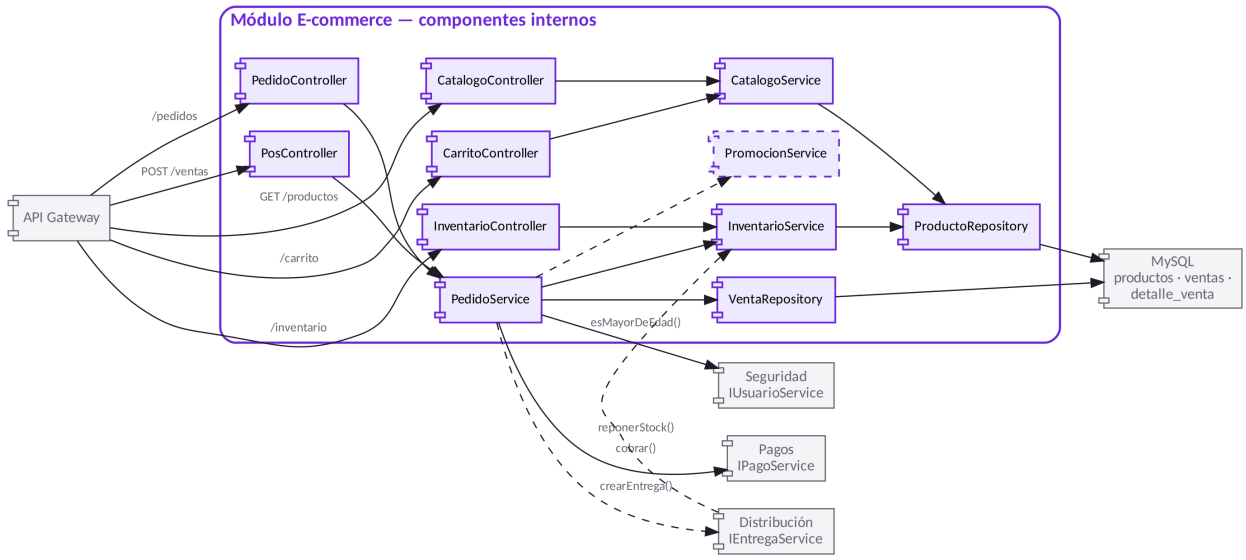


Figura 3. Componentes del módulo E-commerce.

Componentes

Componente	Capa	Responsabilidad
CatalogoController	Interfaz	Lista pública, detalle y mantenimiento de productos.
PedidoController	Interfaz	Checkout, mis pedidos y seguimiento.
VentaMostradorController	Interfaz	Venta POS para el cajero.
InventarioController	Interfaz	Entradas y ajustes de stock.
CatalogoService	Lógica	Búsqueda, filtros y alta/baja de productos.
CarritoService	Lógica	Totales y validación del carrito.
PedidoService	Lógica	Casos de checkout y de venta; coordina la transacción.
InventarioService	Lógica	Verifica y descuenta stock con bloqueo; repone.
PromocionService	Lógica	Calcula descuentos vigentes (propuesto).
EstadoPedido	Lógica	Máquina de estados de la venta (State).
ProductoRepository / VentaRepository	Datos	Persistencia de productos, ventas y detalle.

Reglas de negocio

- El stock nunca es negativo: se verifica y descuenta en la misma transacción con `SELECT … FOR UPDATE`.
- El precio de la venta se copia a `detalle_venta.precio_unitario`; los cambios posteriores no alteran ventas pasadas.
- El delivery exige mayoría de edad, dirección y teléfono.
- El cliente solo puede crear y ver pedidos a su nombre.
- La venta de mostrador nace pagada; el delivery nace pendiente.
- Un producto no se borra si tiene ventas: se desactiva.
- Un pedido pendiente sin atender se cancela y devuelve el stock tras un plazo configurable (propuesto).

Datos

Tablas `productos`, `ventas` y `detalle_venta` (existentes); `categorias`, `promociones` y `movimientos_stock` (propuestas, anexo B).

Patrones aplicados

- **State**: *EstadoPedido* define transiciones válidas (pendiente → en_camino → entregada, etc.).
- **Repository**: aísla el SQL de los servicios.
- **Service Layer**: la transacción vive en *PedidoService*, no en el controlador.
- **Facade** *EcommerceFacade*: `crearVenta()`, `reponerStock()`, `marcarPagada()`.

Endpoints principales

Método y ruta	Rol	Descripción
GET /api/v1/productos	Público	Catálogo activo con stock.
POST|PUT|DELETE /api/v1/productos	Administrador	Mantenimiento (el cajero solo consulta).
POST /api/v1/pedidos	Cliente	Checkout de delivery.
GET /api/v1/pedidos	Cliente	Mis pedidos (filtrado por propietario).
POST /api/v1/ventas	Cajero	Venta de mostrador.
GET /api/v1/ventas	Administrador, cajero	Listado con filtros.
POST /api/v1/inventario/movimientos	Bodega, administrador	Entrada o ajuste de stock.

7.8 Módulo Pagos

Cobra lo vendido, sea en efectivo o por QR, controla la caja del turno y emite el comprobante. Aísla la dependencia con el banco para que el resto del sistema no conozca los detalles del proveedor.

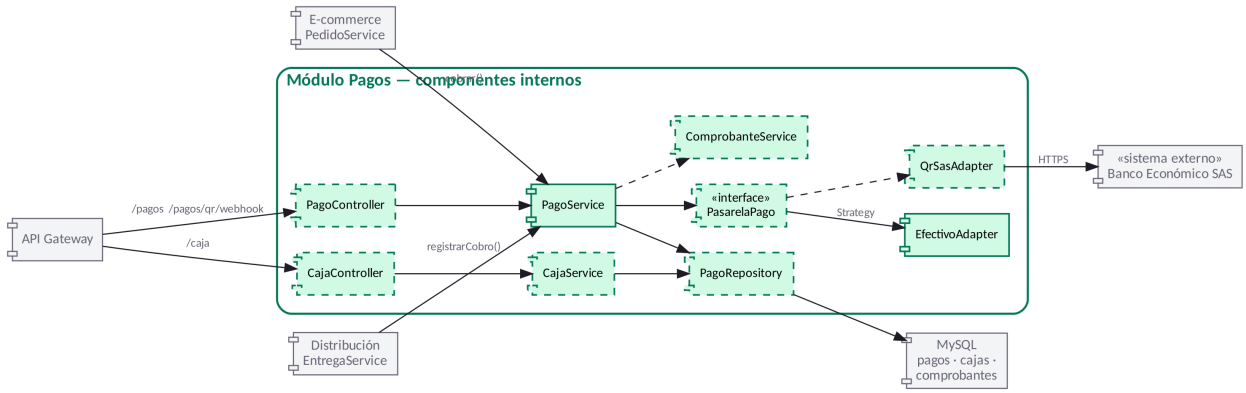


Figura 4. Componentes del módulo Pagos.

Componentes

Componente	Capa	Responsabilidad
PagoController	Interfaz	Registrar pago, consultar estado.
WebhookController	Interfaz	Recibe la notificación del banco.
CajaController	Interfaz	Apertura, movimientos y cierre.
ComprobanteController	Interfaz	Emisión y reimpresión.
PagoService	Lógica	Orquesta el cobro con la estrategia elegida.
PasarelaPago	Lógica	Interfaz Strategy con *PagoEfectivo* y *PagoQr*.
QrSasAdapter	Lógica	Traduce al servicio SAS del Banco Económico.
CajaService	Lógica	Arqueo y diferencias.
ComprobanteService	Lógica	Genera el comprobante numerado.
PagoRepository / CajaRepository	Datos	Persistencia.

Reglas de negocio

- Una venta tiene un solo pago confirmado; la referencia de cobro es única (idempotencia).
- El monto del QR es el total exacto de la venta.
- La notificación del banco se acepta solo con firma válida y referencia conocida.
- Un QR vence en 10 minutos; al vencer, la venta pendiente se anula y el stock vuelve.
- La caja solo opera con un turno abierto; el cierre calcula esperado menos contado.
- Un pago confirmado no se edita: se corrige con un movimiento inverso.

Datos

Tablas propuestas: `pagos`, `cajas`, `movimientos_caja`, `comprobantes`. En el prototipo, el cobro se resume en `ventas.estado_pago` y `ventas.monto_cobrado`.

Patrones aplicados

- **Strategy**: *PasarelaPago* permite agregar tarjeta u otro banco sin modificar *PagoService*.
- **Adapter**: *QrSasAdapter* encapsula el contrato externo.
- **Observer**: publica *PagoConfirmado*.
- **Facade** *PagosFacade*: `cobrar()`, `registrarCobroEntrega()`.

Endpoints principales

Método y ruta	Rol	Descripción
POST /api/v1/pagos	Cajero, cliente	Inicia un cobro (efectivo o QR) para una venta.
GET /api/v1/pagos/{id}	Propietario, cajero	Estado del pago (para consulta periódica).
POST /api/v1/pagos/webhook/qr	Banco (firma)	Confirmación de pago recibido.
POST /api/v1/cajas/apertura	Cajero	Abre el turno con monto inicial.
POST /api/v1/cajas/{id}/cierre	Cajero	Cierra y calcula diferencia.
GET /api/v1/comprobantes/{id}	Propietario, cajero	Comprobante imprimible.

7.9 Módulo Distribución

Lleva los pedidos de delivery desde la bodega hasta el cliente. Registra quién entregó, cuándo, qué ocurrió y cuánto se cobró, y cuida que un pedido fallido no deje stock perdido.

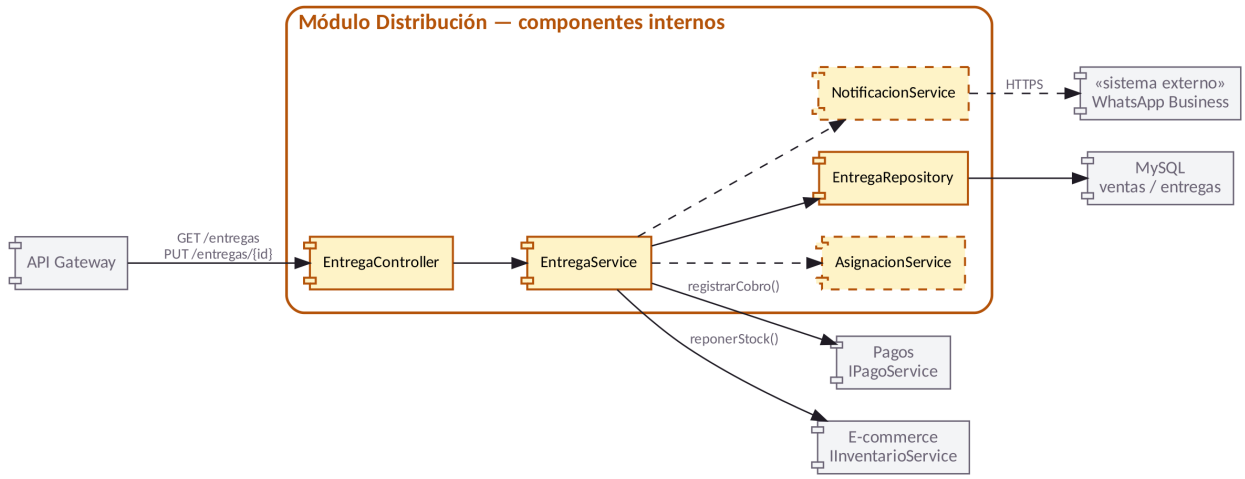


Figura 5. Componentes del módulo Distribución.

Componentes

Componente	Capa	Responsabilidad
EntregaController	Interfaz	Lista de pedidos del repartidor y registro de resultado.
AsignacionController	Interfaz	Asignación manual por el administrador.
EntregaService	Lógica	Transición de estado con transacción y bloqueo.
AsignacionService	Lógica	Reparto de pedidos entre repartidores.
NotificacionService	Lógica	Avisa al cliente por WhatsApp (propuesto).
EntregaRepository	Datos	Persistencia de entregas.

Reglas de negocio

- Solo pedidos *pendiente* o *en_camino* pueden cerrarse; un pedido ya cerrado responde «ya fue procesado» sin cambiar nada.
- Entregado: pago *pagado* y monto cobrado igual al total.
- No entregado: pago *anulado*, monto cobrado cero y stock repuesto **una sola vez**.
- El repartidor solo ve los pedidos que le corresponden.
- Cada cierre guarda repartidor, fecha y observación.

Datos

Columnas de `ventas` (`repartidor_id`, `fecha_entrega`, `observacion_entrega`) en el prototipo; tabla `entregas` propuesta para guardar intentos y geolocalización.

Patrones aplicados

- **State**: transiciones del pedido.
- **Facade** *DistribucionFacade*.
- **Observer**: publica *PedidoEntregado* y *PedidoNoEntregado*.

Endpoints principales

Método y ruta	Rol	Descripción
GET /api/v1/entregas	Repartidor	Pedidos por entregar (propios o sin asignar).
PUT /api/v1/entregas/{id}	Repartidor	Registra «entregado» o «no_entregado» con observación.
GET /api/v1/entregas/historial	Repartidor	Entregas realizadas.
PUT /api/v1/pedidos/{id}/asignacion	Administrador	Asigna un repartidor.

7.10 Diseño de la API REST

La API es el contrato entre los clientes y el servicio. Se diseña según los principios REST: recursos identificados por URL, verbos HTTP con significado, respuestas sin estado y representaciones JSON.

Convenciones

Tema	Convención
Base y versión	`https://api.licofast.bo/api/v1`. Un cambio incompatible crea `/api/v2`.
Recursos	Sustantivos en plural y minúsculas: `/productos`, `/pedidos`, `/pagos`.
Verbos	GET lee (seguro e idempotente), POST crea, PUT reemplaza, PATCH modifica parcialmente, DELETE elimina (baja lógica).
Formato	JSON UTF-8; fechas ISO 8601; dinero como decimal con dos cifras (en bolivianos).
Autenticación	Cabecera `Authorization: Bearer <jwt>`.
Idempotencia	POST de pago y de pedido acepta `Idempotency-Key`; una repetición devuelve el resultado original.
Paginación	`?page=1&size=20`; la respuesta incluye total y páginas.
CORS	Solo los orígenes de LicoFast (no *).
Límite de tasa	60 peticiones por minuto por IP; 5 por minuto en login (Nginx).

Códigos de estado

Código	Uso
200 / 201 / 204	Lectura o modificación exitosa / recurso creado / sin contenido.
400 / 422	Petición mal formada / datos válidos en forma pero rechazados por una regla (edad, stock).
401 / 403	Sin token o token inválido / rol o propiedad insuficiente.
404 / 409	No existe / conflicto (duplicado, pedido ya procesado).
429 / 500 / 503	Límite excedido / error interno sin detalle al cliente / dependencia externa caída.

Formato de error uniforme

```
{
  "error": {
    "codigo": "STOCK_INSUFICIENTE",
    "mensaje": "No hay stock suficiente de Ron anejo 750 ml.",
    "detalle": [{"producto_id": 1, "disponible": 2, "solicitado": 5}],
    "traza": "c1f0a9e2"
  }
}
```

Contenido del token (JWT)

```
{ "sub": 12, "rol": "cajero", "iat": 1761930000, "exp": 1761958800, "jti": "9a1b..." }
// Firma HS256 con secreto de entorno (no en el repositorio). Vence en 8 horas.
```

Ejemplos

Login

```
POST /api/v1/auth/login
{ "usuario": "cajero1", "password": "********" }

200 OK
{ "token": "eyJhbGciOi...", "usuario": { "id": 2, "nombre": "Carla", "rol": "cajero" } }
```

Checkout de delivery

```
POST /api/v1/pedidos          Idempotency-Key: 7d1c...
{ "items": [ {"producto_id": 1, "cantidad": 2}, {"producto_id": 4, "cantidad": 1} ],
  "direccion_entrega": "Calle Bolívar 123", "referencia": "Portón verde",
  "telefono_contacto": "70000010" }

201 Created
{ "id": 58, "estado": "pendiente", "estado_pago": "pendiente", "total": 126.00 }
```

Cierre de entrega

```
PUT /api/v1/entregas/58
{ "accion": "no_entregado", "observacion": "El cliente no respondió" }

200 OK
{ "id": 58, "estado": "no_entregado", "estado_pago": "anulado", "stock_repuesto": true }
```

Correspondencia entre las rutas actuales y las rutas objetivo

Ruta actual (prototipo)	Ruta objetivo	Cambio
POST /api/login	POST /api/v1/auth/login	Versión y agrupación; token firmado.
POST /api/register	POST /api/v1/auth/register	Versión; edad exacta.
/api/usuarios y /api/clientes (alias)	/api/v1/usuarios	Se elimina el alias duplicado.
GET /api/productos-publicos	GET /api/v1/productos	Misma lista; sin `SELECT *`.
/api/productos (POST, PUT, DELETE)	/api/v1/productos	DELETE solo administrador y baja lógica.
POST /api/ventas (cliente o cajero)	POST /api/v1/pedidos y POST /api/v1/ventas	Se separan delivery y mostrador; se ignora `cliente_id` enviado por el cliente.
GET /api/entregas y PUT /api/entregas/{id}	/api/v1/entregas	Se valida que el pedido esté abierto antes de reponer stock.
GET /api/health	GET /api/v1/health	Se añade estado de la base de datos.

7.11 Vista de datos

El modelo de datos relacional (figura en 8.5) tiene cuatro tablas implementadas y nueve propuestas. La clase de cada tabla pertenece a un solo módulo; los demás la consultan mediante la fachada. Las reglas de integridad se aplican en la base de datos siempre que sea posible, y en el servicio cuando involucran varias tablas.

#	Invariante	Dónde se garantiza
I-1	El stock de un producto nunca es negativo.	Servicio (bloqueo de fila) y restricción `CHECK (stock >= 0)`.
I-2	El total de una venta es la suma de cantidad × precio unitario de su detalle.	Servicio, dentro de la transacción.
I-3	Una venta entregada tiene `estado_pago = pagado` y monto cobrado igual al total.	Servicio de entregas.
I-4	Una venta no entregada tiene `estado_pago = anulado` y monto cobrado cero.	Servicio de entregas.
I-5	Un producto con ventas no puede borrarse.	Clave foránea `ON DELETE RESTRICT` en `detalle_venta`.
I-6	Usuario y CI son únicos.	Índices únicos.
I-7	Un pago confirmado tiene referencia única.	Índice único en `pagos.referencia`.
I-8	La suma de movimientos de stock explica el stock actual.	Tabla `movimientos_stock` y reporte de conciliación.

7.12 Vista de comportamiento

Los diagramas de secuencia y de estados se presentan en las secciones 8.6 y 8.7. Esta tabla resume qué muestra cada uno y qué decisiones de diseño contiene.

Diagrama	Escenario	Decisión que ilustra
Secuencia 1	Inicio de sesión y emisión del JWT.	El hash se compara en el servicio; el controlador nunca ve la contraseña almacenada.
Secuencia 2	Checkout de un pedido de delivery.	Una sola transacción: verificar, descontar, crear venta y detalle; si algo falla, se revierte todo.
Secuencia 3	Venta en mostrador con pago por QR.	La confirmación llega por *webhook*; el cajero ve el resultado por consulta periódica.
Secuencia 4	Entrega o no entrega.	El cierre bloquea el pedido y repone stock solo si estaba abierto.
Estados 1	Ciclo de vida de `ventas.estado`.	Cuatro transiciones válidas; *entregada* y *no_entregado* son finales.
Estados 2	Ciclo de `estado_pago`.	*pagado* y *anulado* son finales; solo se llega a *pagado* con un cobro confirmado.

7.13 Patrones de diseño aplicados

Los diagramas de clases de cada patrón están en la sección 8.2. Se usan solo donde resuelven un problema concreto del proyecto.

Patrón	Problema en LicoFast	Solución	Módulo
Front Controller	Autenticar y autorizar en cada endpoint repite código y se olvida.	Un punto de entrada con middleware JWT y guardas de rol.	**Seguridad**
Service Layer	La transacción de venta está dentro del endpoint y se duplica en PHP, Python y Java.	Servicios con la lógica; los controladores solo traducen HTTP.	Todos
Repository	SQL disperso dificulta cambiar el esquema y probar.	Una clase por agregado con consultas parametrizadas.	Todos
Strategy	Efectivo y QR (y mañana tarjeta) se cobran distinto.	Interfaz *PasarelaPago* con una clase por método.	**Pagos**

Patrón	Problema en LicoFast	Solución	Módulo
Adapter	El contrato del banco es externo y puede cambiar.	*QrSasAdapter* traduce a un contrato propio estable.	Pagos
State	Las transiciones del pedido se validan con condiciones sueltas.	*EstadoPedido* define qué cambios son válidos.	E-commerce
Observer	Auditar y notificar no debe enredarse con la lógica de venta.	Un *EventBus* interno; los observadores se suscriben.	Todos
Facade	Un módulo no debe conocer las clases internas de otro.	Una fachada por módulo con operaciones de negocio.	Todos

7.14 Arquitectura de seguridad

La seguridad es el primer atributo de calidad (7.2). Se organiza en defensa por capas: cada capa asume que la anterior pudo fallar.

Controles según el riesgo (referencia OWASP Top 10)

Riesgo	Control en la arquitectura	Estado
Control de acceso roto	Rol exigido por endpoint (RbacGuard) y comprobación de propiedad: el cliente solo accede a sus recursos.	Parcial
Fallas criptográficas	HTTPS obligatorio; PBKDF2-SHA256 con sal; secreto JWT en variable de entorno; datos sensibles fuera de los registros.	Parcial
Inyección	Consultas parametrizadas en todas las capas; validación de entrada por esquema.	Presente
Diseño inseguro	Verificación de edad en el servidor; transacciones; firma y monto en el webhook.	Parcial
Configuración incorrecta	Sin *debug* en producción; CORS restringido; usuario de base sin privilegios de administración; cabeceras de seguridad en Nginx.	Pendiente
Componentes vulnerables	Versiones fijadas y revisión con `pip-audit`.	Parcial
Autenticación débil	Límite de intentos, bloqueo temporal, token con vencimiento, contraseñas sin valores por defecto.	Pendiente
Integridad de datos y software	Idempotencia, firma de webhooks, respaldos probados.	Pendiente
Registro y monitoreo	Tabla de auditoría con usuario, acción y fecha; registros de aplicación con identificador de traza.	Pendiente
SSRF	La API solo llama a destinos fijos (banco y WhatsApp).	Aplicable en integración

Matriz de permisos por rol (RBAC)

C = crear, **L** = leer, **M** = modificar, **E** = eliminar (baja), **P** = solo lo propio, — = sin acceso.

Recurso	Visitante	Cliente	Cajero	Bodega	Repartidor	Admin.
Catálogo de productos	L	L	L	L	—	C L M E
Stock e inventario	—	—	L	C L M	—	C L M
Pedido de delivery	—	C, L P	L	—	L (asignados)	L M
Venta de mostrador	—	—	C L	—	—	L
Pago y comprobante	—	C, L P	C L	—	C (cobro entrega)	L

Recurso	Visitante	Cliente	Cajero	Bodega	Repartidor	Admin.
Caja	—	—	C L P	—	—	L
Entrega	—	L P	—	—	M	L M
Usuarios y roles	C (registro)	L M P	—	—	—	C L M E
Auditoría	—	—	—	—	—	L

Protección de datos personales

LicoFast guarda nombre, CI, fecha de nacimiento, teléfono y dirección. Se aplica el principio de mínimo dato: no se almacena el número completo de ninguna tarjeta (no se aceptan en esta versión), los respaldos van cifrados y las respuestas de la API nunca incluyen el hash de contraseña.

7.15 Escenarios de atributos de calidad

ID	Atributo	Escenario (fuente, estímulo, respuesta)	Medida
EC-1	Seguridad	Un atacante modifica el rol dentro del token y llama a un endpoint de administrador. El servicio rechaza la firma inválida.	403 o 401 en el 100 % de los intentos
EC-2	Integridad	Dos clientes compran la última botella al mismo tiempo. El primero confirma; el segundo recibe «stock insuficiente».	Stock final = 0, nunca negativo
EC-3	Integridad	El banco envía dos veces la misma confirmación de pago. El sistema registra un solo cobro.	1 pago por referencia
EC-4	Rendimiento	El cajero registra una venta de 10 productos en hora pico con 20 usuarios conectados.	Menos de 1 s (percentil 95)
EC-5	Disponibilidad	El proceso de aplicación se cae. El supervisor lo reinicia.	Servicio restablecido en menos de 30 s
EC-6	Disponibilidad	El banco no responde al generar el QR. El cajero cobra en efectivo.	Venta posible sin QR
EC-7	Modificabilidad	El banco cambia su contrato. Se modifica solo *QrSasAdapter*.	Un archivo cambiado; pruebas de pago aprobadas
EC-8	Modificabilidad	Se agrega pago con tarjeta creando una nueva estrategia.	Sin cambios en *PagoService*
EC-9	Recuperabilidad	Se corrompe la base de datos. Se restaura el último respaldo.	Pérdida máxima de datos: 24 h; restauración en menos de 2 h
EC-10	Usabilidad	Un cajero nuevo registra su primera venta con una guía de una página.	Menos de 5 minutos

Las medidas son propuestas de diseño que deben ratificarse con el cliente.

7.16 Decisiones arquitectónicas (ADR)

Cada decisión se registra con su contexto y sus consecuencias, para que quien lea el proyecto después sepa por qué se hizo así.

ID	Decisión	Motivo	Consecuencia
ADR-01	Monolito modular (un solo servicio con cuatro módulos).	Transacciones atómicas, equipo pequeño, costo bajo.	Un único despliegue; los módulos podrían extraerse más adelante.
ADR-02	Backend en Python con Flask. Java queda como alternativa; PHP como prototipo de desarrollo.	Experiencia del equipo, poco código, buen soporte REST.	Se retiran las lógicas duplicadas en PHP y Java cuando Flask alcance la paridad.

ID	Decisión	Motivo	Consecuencia
ADR-03	API REST versionada (`/api/v1`) con JSON.	Un contrato único para todos los clientes.	Los cambios incompatibles exigen una nueva versión.
ADR-04	Autenticación con JWT firmado (HS256) y vencimiento.	Sin estado, simple; sustituye al token «id|rol».	No se puede revocar un token antes de su vencimiento: se usa vida corta y lista de bloqueo opcional.
ADR-05	Mantener PBKDF2-SHA256 con sal por usuario (≥ 120 000 iteraciones; objetivo 600 000).	Ya implementado y compatible entre PHP, Python y Java.	Formato versionado en el campo, para migrar a otro algoritmo sin invalidar las claves.
ADR-06	MySQL 8 con InnoDB como única base de datos.	Transacciones, XAMPP y RDS.	No hay almacenamiento no relacional. XAMPP incluye MariaDB, compatible con el esquema, para el entorno de desarrollo.
ADR-07	Transacciones con `SELECT ... FOR UPDATE` para stock y entregas.	Evitar ventas dobles y reposición doble.	Bloqueos cortos; el orden de bloqueo por id evita interbloqueos.
ADR-08	Pago QR detrás de Strategy y Adapter, con *webhook* idempotente.	El contrato del banco no existe aún.	Se puede simular el banco en pruebas.
ADR-09	Eventos internos (Observer) para auditoría y notificaciones.	Desacoplar efectos secundarios.	Sin cola externa en esta versión; si falla un observador no se revierte la venta.
ADR-10	Baja lógica en usuarios y productos.	Conservar el historial de ventas.	Todas las consultas filtran por `activo`.
ADR-11	Despliegue en AWS (EC2, RDS, S3) con Nginx y TLS.	Servicios administrados, respaldo y escalado vertical.	Costo mensual fijo; sustituible por un VPS.
ADR-12	Tres roles de personal más administrador; «empleado» del código equivale a bodega.	Coincide con el esquema actual y con la Visión.	Se documenta el mapeo para evitar confusión.

7.17 Análisis del código actual frente a la arquitectura

Se revisó el repositorio (backend Python, backend PHP, páginas PHP, esqueleto Java, frontend y SQL). El prototipo ya demuestra los flujos principales, y el registro de ventas y de entregas usa transacciones correctamente. Los hallazgos siguientes se refieren a lo que hay que cambiar antes de operar con clientes reales. **La implementación de referencia (carpeta LicoFastFinal del repositorio) ya los corrige; ver el anexo E.**

ID	Hallazgo	Severidad	Recomendación
H-01	El token es `"id|rol"`, sin firma ni vencimiento: cualquiera puede escribirlo y hacerse pasar por administrador.	Crítica	JWT HS256 con secreto de entorno y vencimiento (ADR-04).
H-02	`POST /api/ventas` acepta `cliente_id` de cualquier usuario: un cliente puede registrar pedidos a nombre de otro.	Alta	Ignorar el campo salvo para el cajero; usar la identidad del token.
H-03	En «no entregado» el stock se repone sin comprobar que la actualización afectó una fila: repetir la acción suma el stock varias veces. La rama «entregado» tampoco informa si no cambió nada.	Alta	Revisar `rowcount` o bloquear la fila y responder 409 si el pedido ya está cerrado.
H-04	CORS abierto a cualquier origen; `debug=True` y escucha en `0.0.0.0`.	Alta	Restringir orígenes; desactivar *debug*; Gunicorn detrás de Nginx.
H-05	Conexión a MySQL con `root` sin contraseña, escrita en el código. Los usuarios semilla comparten la clave `Admin1234` y la misma sal.	Alta	Variables de entorno; usuario de base con privilegios mínimos; no cargar datos semilla en producción.

ID	Hallazgo	Severidad	Recomendación
H-06	El administrador crea usuarios con la contraseña por defecto `Usuario123` si no la indica.	Media	Exigir contraseña o generar una temporal con cambio obligatorio.
H-07	Sin límite de intentos de login ni bloqueo.	Alta	Límite de tasa en Nginx y bloqueo por cuenta (CU-S02).
H-08	El stock reservado por un pedido pendiente nunca caduca.	Media	Cancelación automática tras un plazo configurable.
H-09	La edad se calcula con `días // 365` (falla en años bisiestos) y `POST /ventas` no verifica la mayoría de edad del cliente.	Media	Calcular con la fecha exacta y verificar en el checkout.
H-10	Borrado físico de productos; el cajero también puede borrarlos y modificarlos. El borrado de usuarios depende de capturar un error de la base.	Media	Baja lógica; permisos por rol según la matriz 7.14.
H-11	Una conexión por consulta, sin *pool*; los errores devuelven `str(ex)` al cliente; campos faltantes producen error 500.	Media	Pool, validación de entrada y formato de error uniforme (7.10).
H-12	La misma lógica existe en PHP, Python y Java con diferencias.	Media	Una sola implementación de referencia (ADR-02).
H-13	Sin versionado de la API, sin auditoría y consultas con `SELECT *`.	Baja	Rutas `/api/v1`, tabla de auditoría y columnas explícitas.

Fortalezas

Lo que ya está bien: contraseñas con PBKDF2 y sal individual y comparación en tiempo constante; consultas parametrizadas; transacciones con bloqueo de fila en la venta; reposición de stock al no entregar; baja lógica de usuarios con historial; estados de venta y de pago modelados con ENUM; separación de rutas por rol en el panel PHP.

8. Modelado UML

Esta sección reúne los diagramas pedidos: casos de uso, clases, componentes, despliegue de nodos y componentes, más el modelo entidad-relación, las secuencias y los estados. En los diagramas, el **trazo continuo** indica lo que ya existe en el código y el **trazo punteado** lo propuesto. El color identifica el módulo: **Seguridad**, **E-commerce**, **Pagos** y **Distribución**.

8.1 Diagramas de casos de uso

Módulo Seguridad

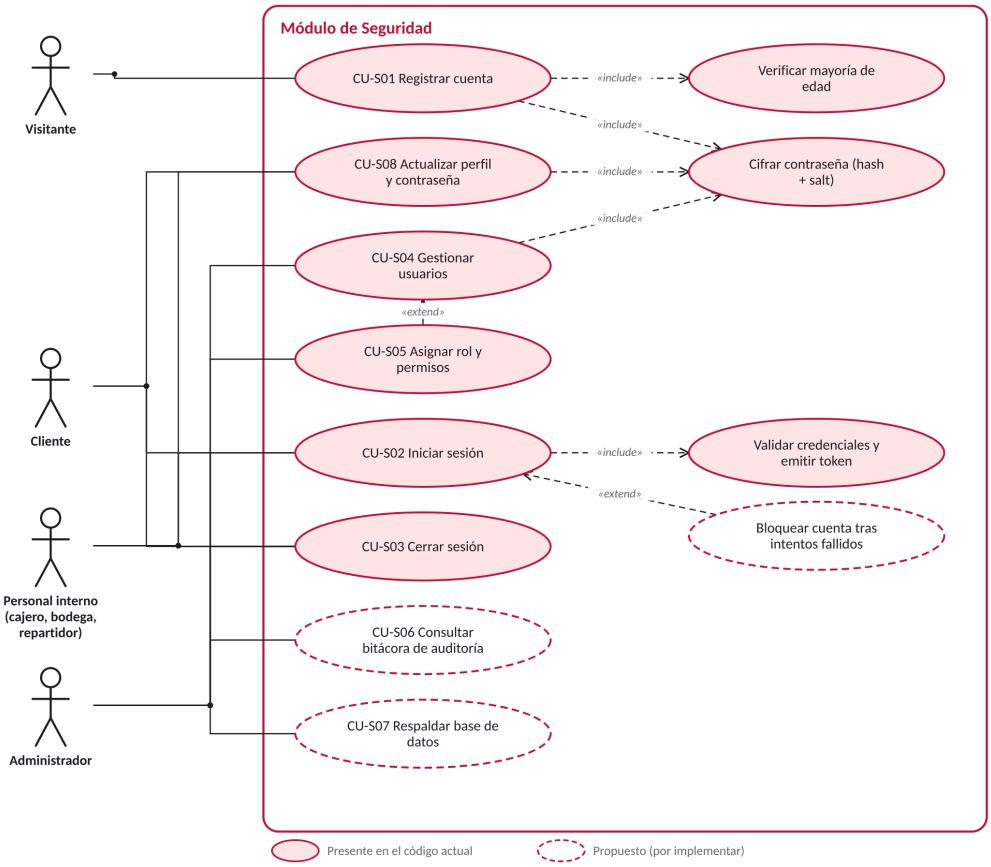


Figura 6. Casos de uso del módulo Seguridad.

ID	Caso de uso	Actor	Req.	HU	Estado
CU-S01	Registrar cuenta	Visitante	RF-16, 17	HU-S01	Implementado
CU-S02	Iniciar sesión	Todos los roles	RF-18, 22	HU-S02	Implementado
CU-S03	Cerrar sesión	Todos los roles	RF-18	HU-S03	Implementado
CU-S04	Gestionar usuarios	Administrador	RF-19	HU-S04	Implementado
CU-S05	Asignar rol y permisos	Administrador	RF-20, 22	HU-S05	Implementado
CU-S06	Consultar bitácora de auditoría	Administrador	RF-25	HU-S06	Propuesto
CU-S07	Respaldar base de datos	Planificador, admin.	RF-26	HU-S07	Propuesto
CU-S08	Actualizar perfil y contraseña	Usuario autenticado	RF-18	HU-S02	Parcial

Módulo E-commerce

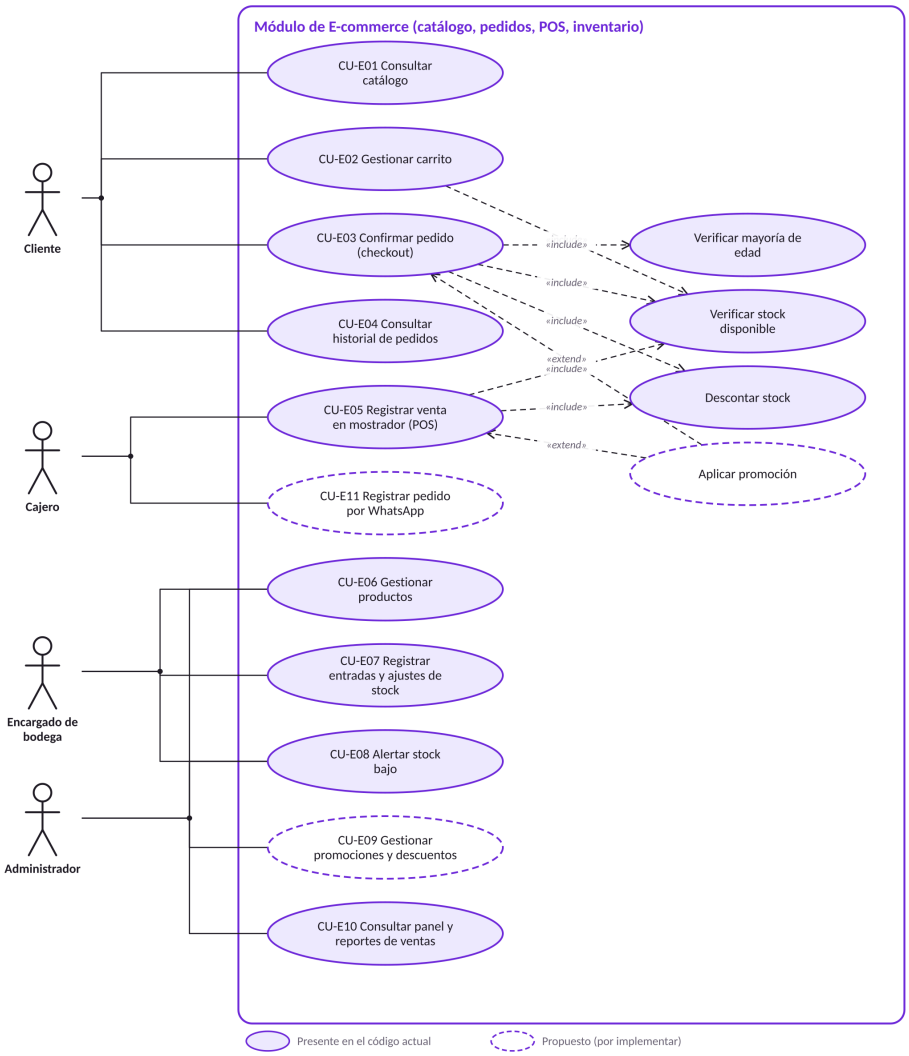


Figura 7. Casos de uso del módulo E-commerce.

ID	Caso de uso	Actor	Req.	HU	Estado
CU-E01	Consultar catálogo	Visitante, cliente	RF-01, 24	HU-E01	Parcial
CU-E02	Gestionar carrito	Cliente	RF-02	HU-E02	Implementado
CU-E03	Confirmar pedido (checkout)	Cliente	RF-02, 06, 16	HU-E03	Implementado
CU-E04	Consultar historial de pedidos	Cliente	RF-09, 15	HU-E04	Implementado
CU-E05	Registrar venta en mostrador	Cajero	RF-03, 06	HU-E05	Implementado
CU-E06	Gestionar productos	Administrador	RF-05, 24	HU-E06	Parcial
CU-E07	Registrar entradas y ajustes de stock	Bodega	RF-05, 23	HU-E07	Propuesto
CU-E08	Alertar stock bajo	Sistema	RF-07	HU-E08	Parcial
CU-E09	Gestionar promociones	Administrador	RF-14	HU-E09	Propuesto
CU-E10	Consultar panel y reportes	Administrador	RF-21	HU-E10	Implementado
CU-E11	Registrar pedido por WhatsApp	Cajero	RF-08	HU-E11	Propuesto

Módulo Pagos

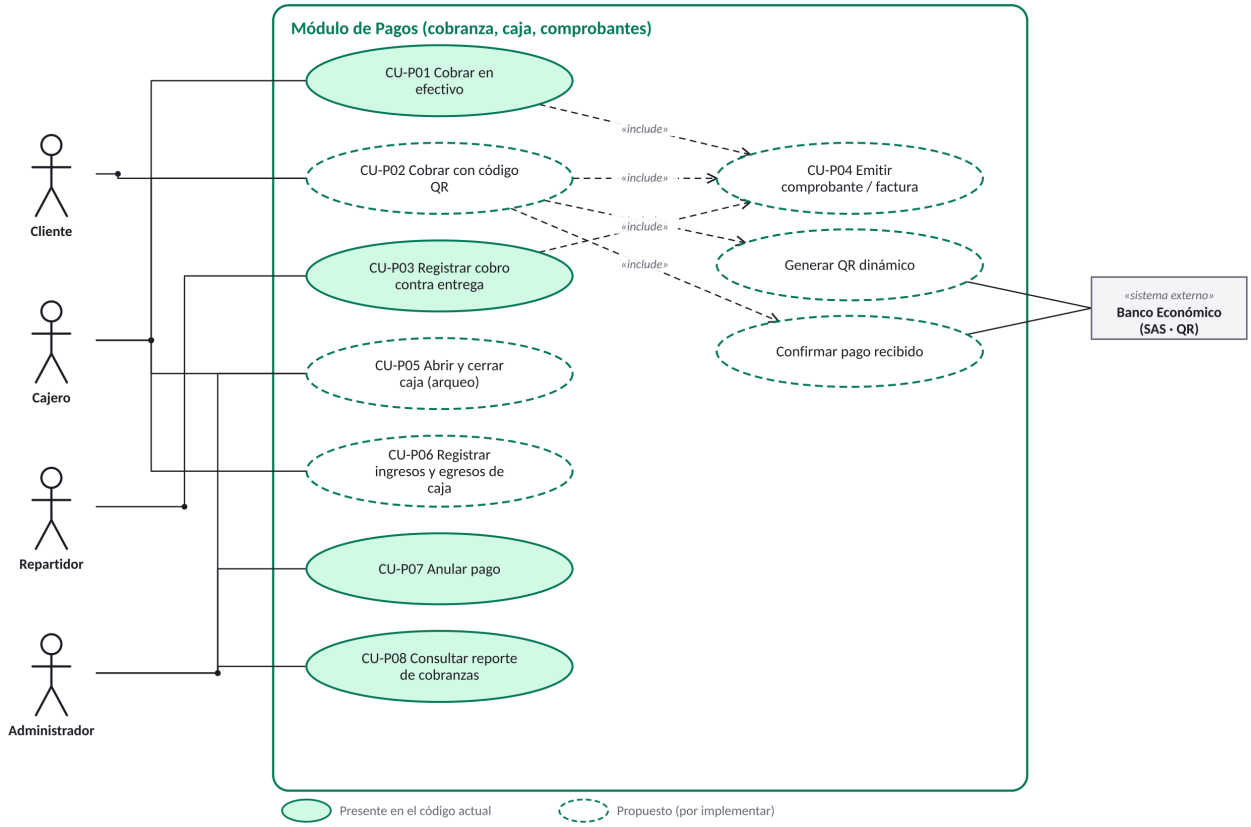


Figura 8. Casos de uso del módulo Pagos.

ID	Caso de uso	Actor	Req.	HU	Estado
CU-P01	Cobrar en efectivo	Cajero	RF-04	HU-P01	Implementado
CU-P02	Cobrar con código QR	Cajero, cliente, banco	RF-04	HU-P02	Propuesto
CU-P03	Registrar cobro contra entrega	Repartidor	RF-04, 11	HU-P03	Implementado
CU-P04	Emitir comprobante o factura	Cajero	RF-13	HU-P04	Propuesto
CU-P05	Abrir y cerrar caja	Cajero	RF-12	HU-P05	Propuesto
CU-P06	Registrar ingresos y egresos	Cajero	RF-12	HU-P05	Propuesto
CU-P07	Anular pago	Administrador	RF-04, 25	HU-P06	Parcial
CU-P08	Reporte de cobranzas	Administrador	RF-21	HU-E10	Parcial

Módulo Distribución

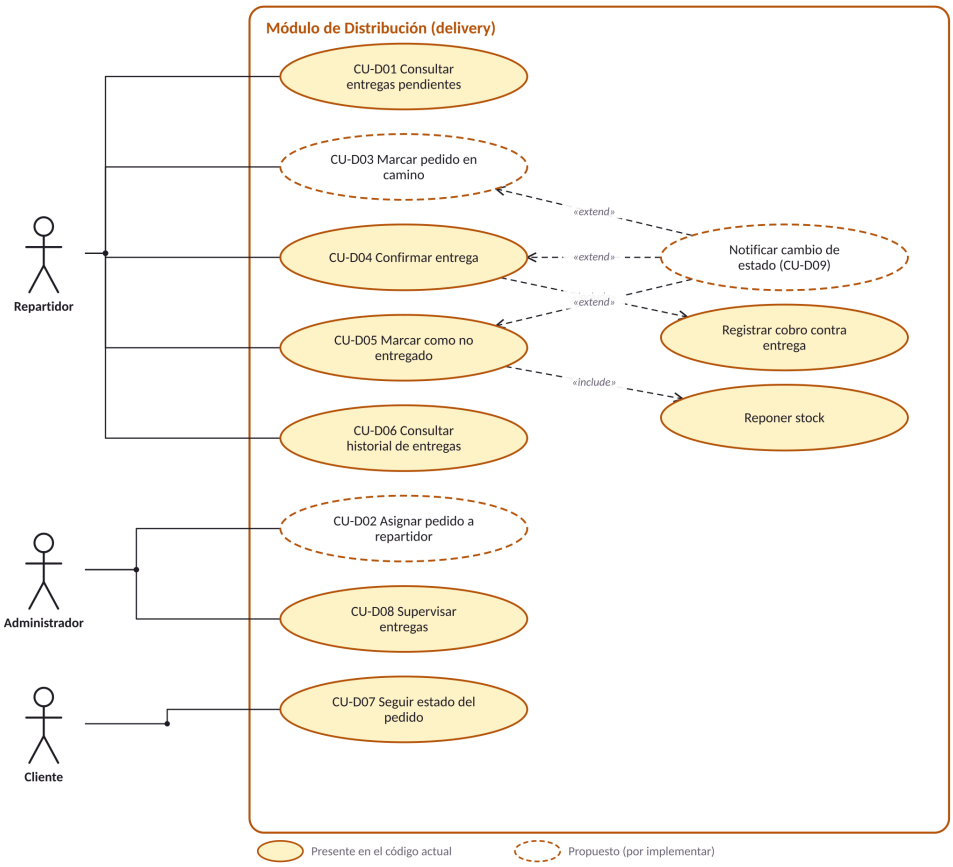


Figura 9. Casos de uso del módulo Distribución.

ID	Caso de uso	Actor	Req.	HU	Estado
CU-D01	Consultar entregas pendientes	Repartidor	RF-10, 11	HU-D01	Implementado
CU-D02	Asignar pedido a repartidor	Administrador	RF-10	HU-D02	Propuesto
CU-D03	Marcar pedido en camino	Repartidor	RF-09, 11	HU-D03	Propuesto
CU-D04	Confirmar entrega	Repartidor	RF-11	HU-D04	Implementado
CU-D05	Marcar como no entregado	Repartidor	RF-06, 11	HU-D05	Implementado
CU-D06	Consultar historial de entregas	Repartidor	RF-11	HU-D06	Implementado
CU-D07	Seguir estado del pedido	Cliente	RF-09	HU-D07	Parcial
CU-D08	Supervisar entregas	Administrador	RF-21	HU-E10	Implementado
CU-D09	Notificar cambio de estado	Sistema, WhatsApp	RF-09	HU-D07	Propuesto

8.2 Diagramas de clases

Modelo de dominio

Las clases son las entidades del negocio; las asociaciones muestran multiplicidad. El color de cada clase indica el módulo dueño. Las clases punteadas son propuestas.

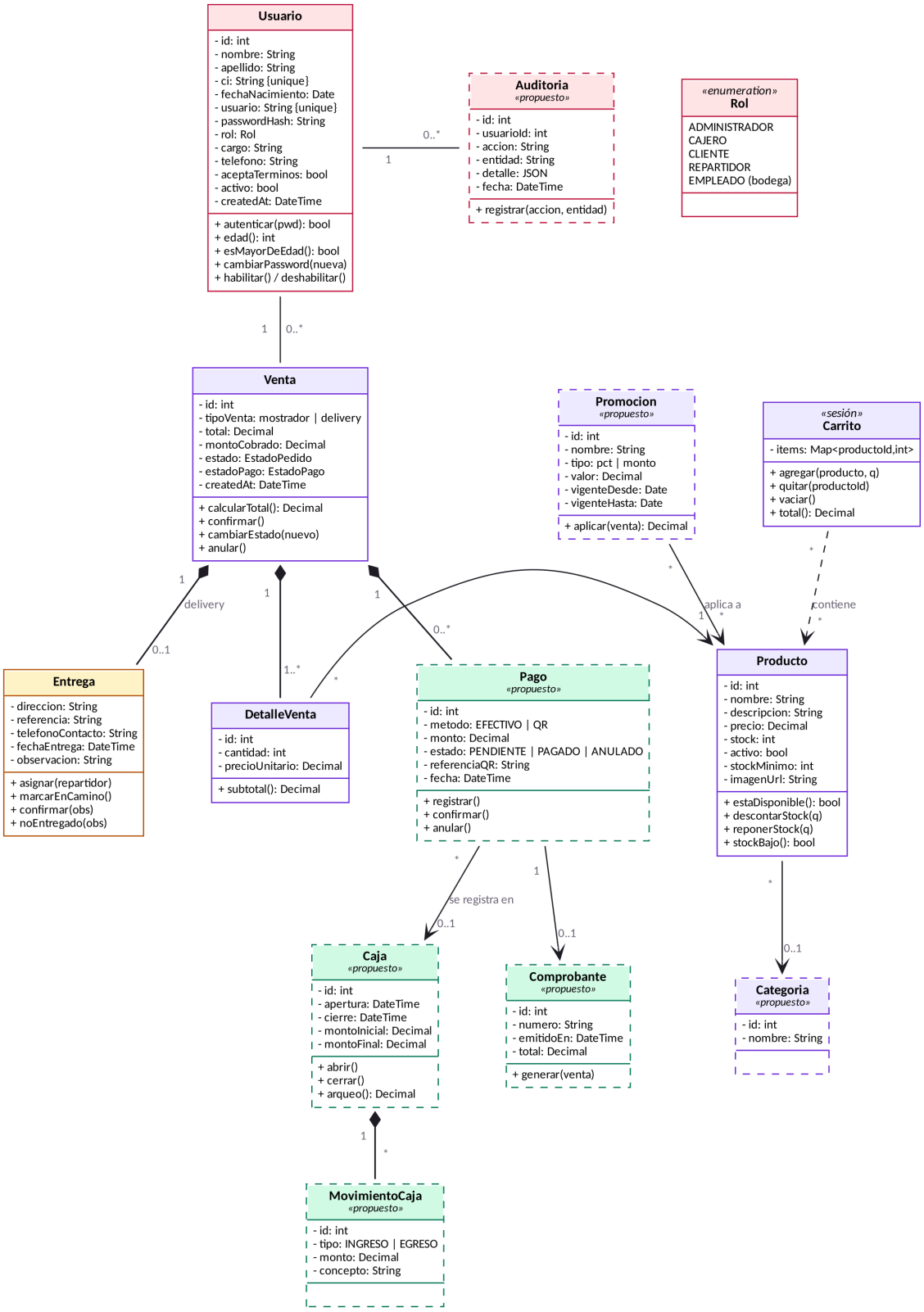


Figura 10. Diagrama de clases del dominio de LicoFast.

Clases de los patrones de diseño

Cada diagrama muestra cómo se aplica un patrón a una parte concreta del sistema (ver 7.13).

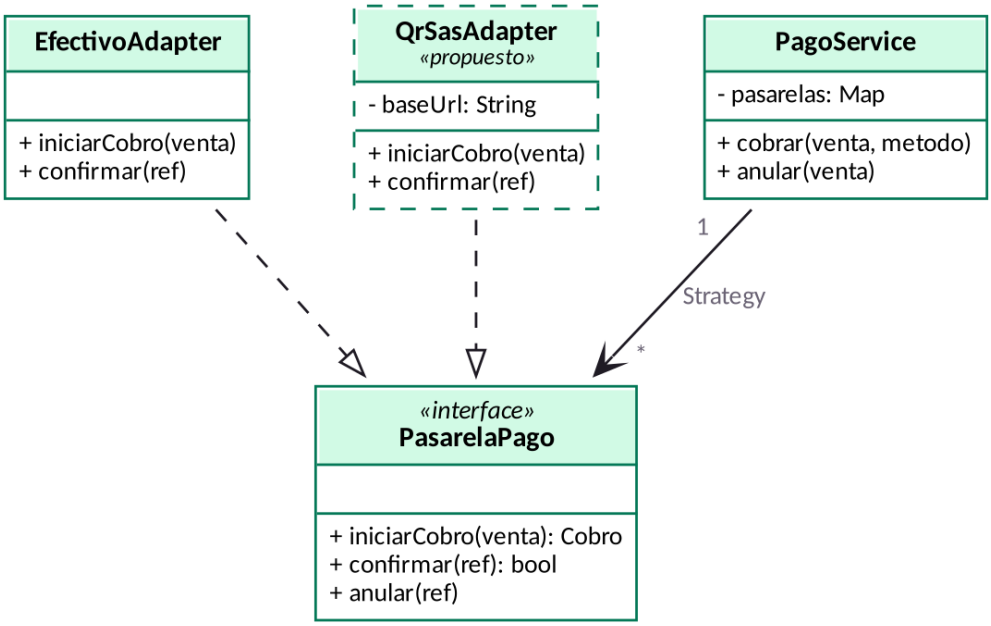


Figura 11. Strategy: formas de pago intercambiables (Pagos).

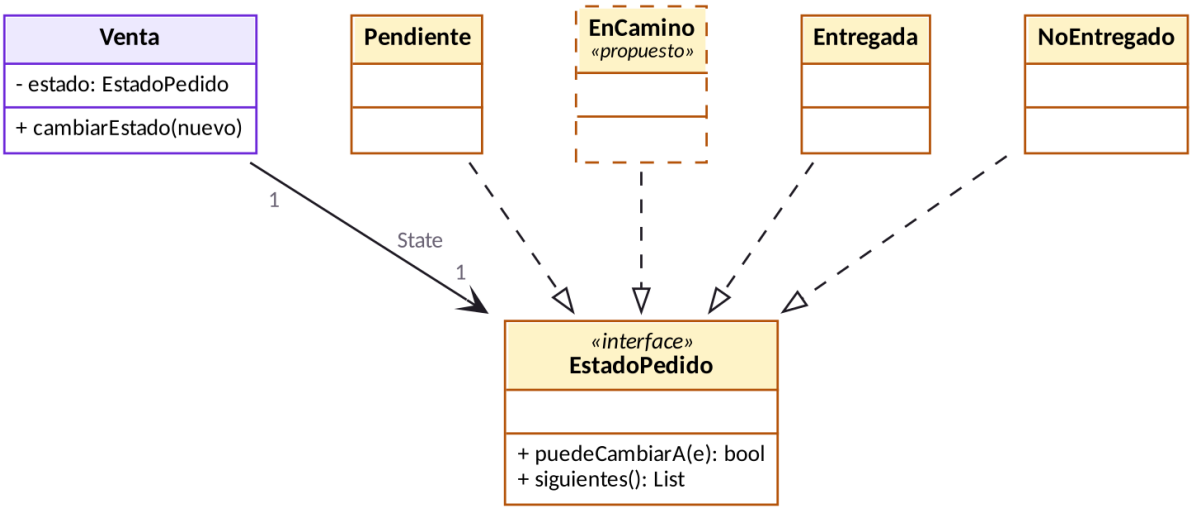


Figura 12. State: ciclo de vida del pedido (E-commerce y Distribución).

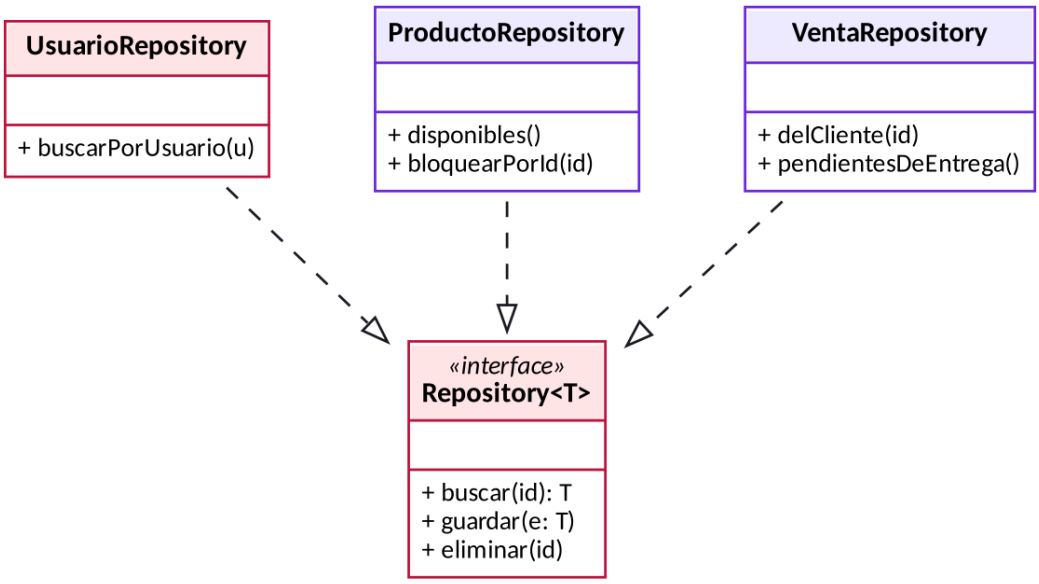


Figura 13. Repository y Service Layer: acceso a datos desacoplado.

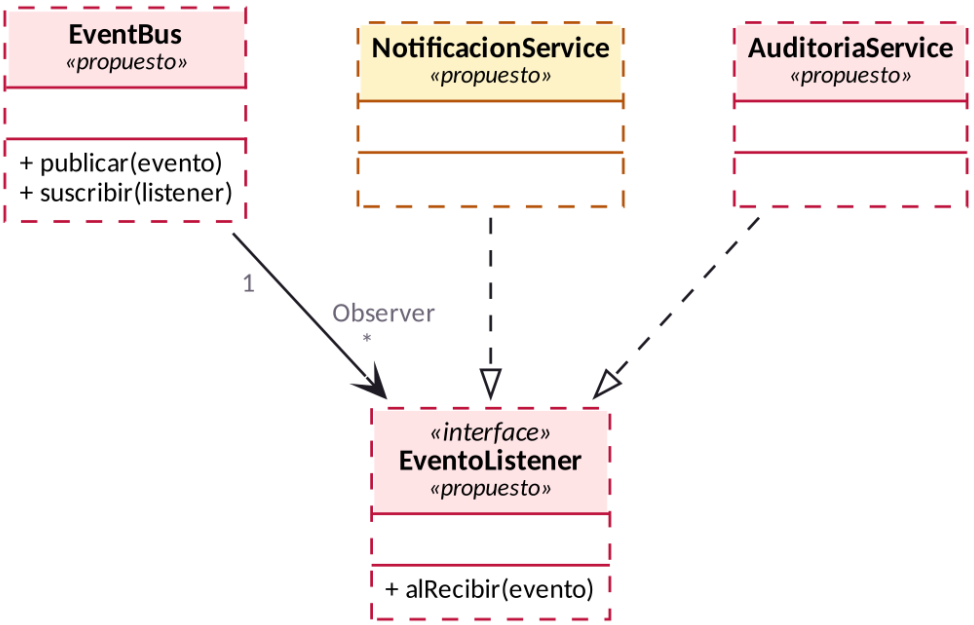


Figura 14. Observer: eventos internos para auditoría y notificaciones.

8.3 Diagrama de componentes

Vista general: el cliente, la puerta de entrada (Nginx y middleware), los cuatro módulos con sus fachadas, la base de datos y los sistemas externos. Los componentes internos de cada módulo están en las secciones 7.6 a 7.9.

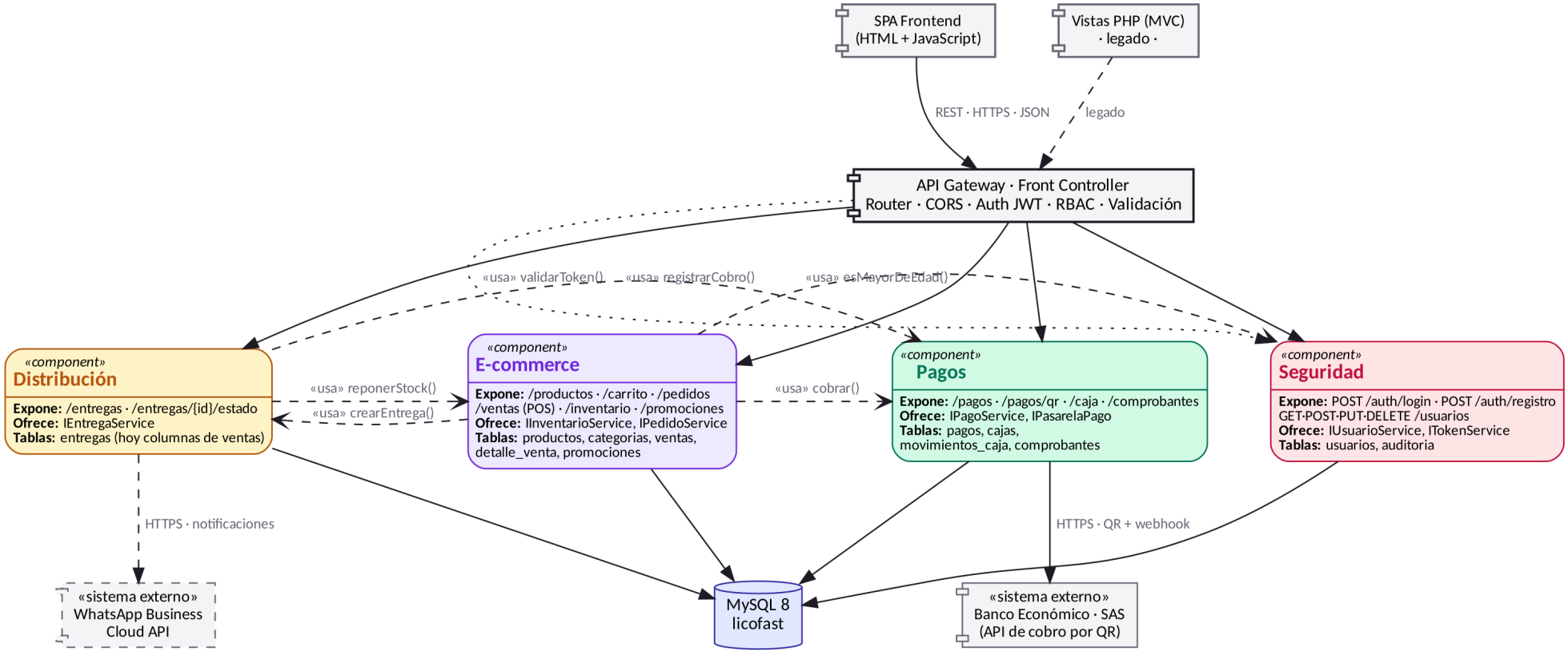


Figura 15. Componentes de LicoFast: cuatro módulos, base de datos y sistemas externos.

8.4 Diagramas de despliegue

Entorno de desarrollo (prototipo actual)

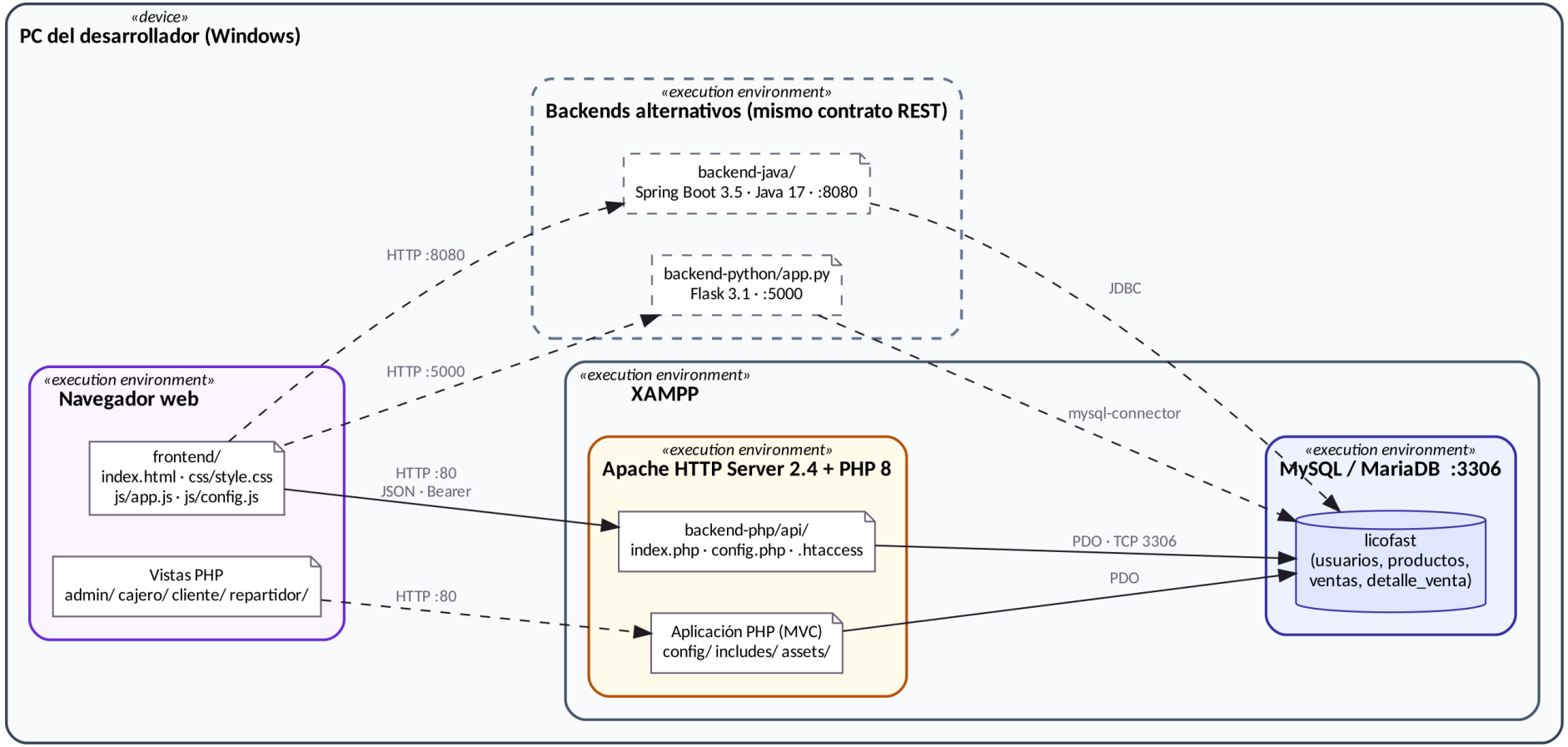


Figura 16. Despliegue de desarrollo: XAMPP en el equipo del estudiante.

Entorno de producción (propuesto)

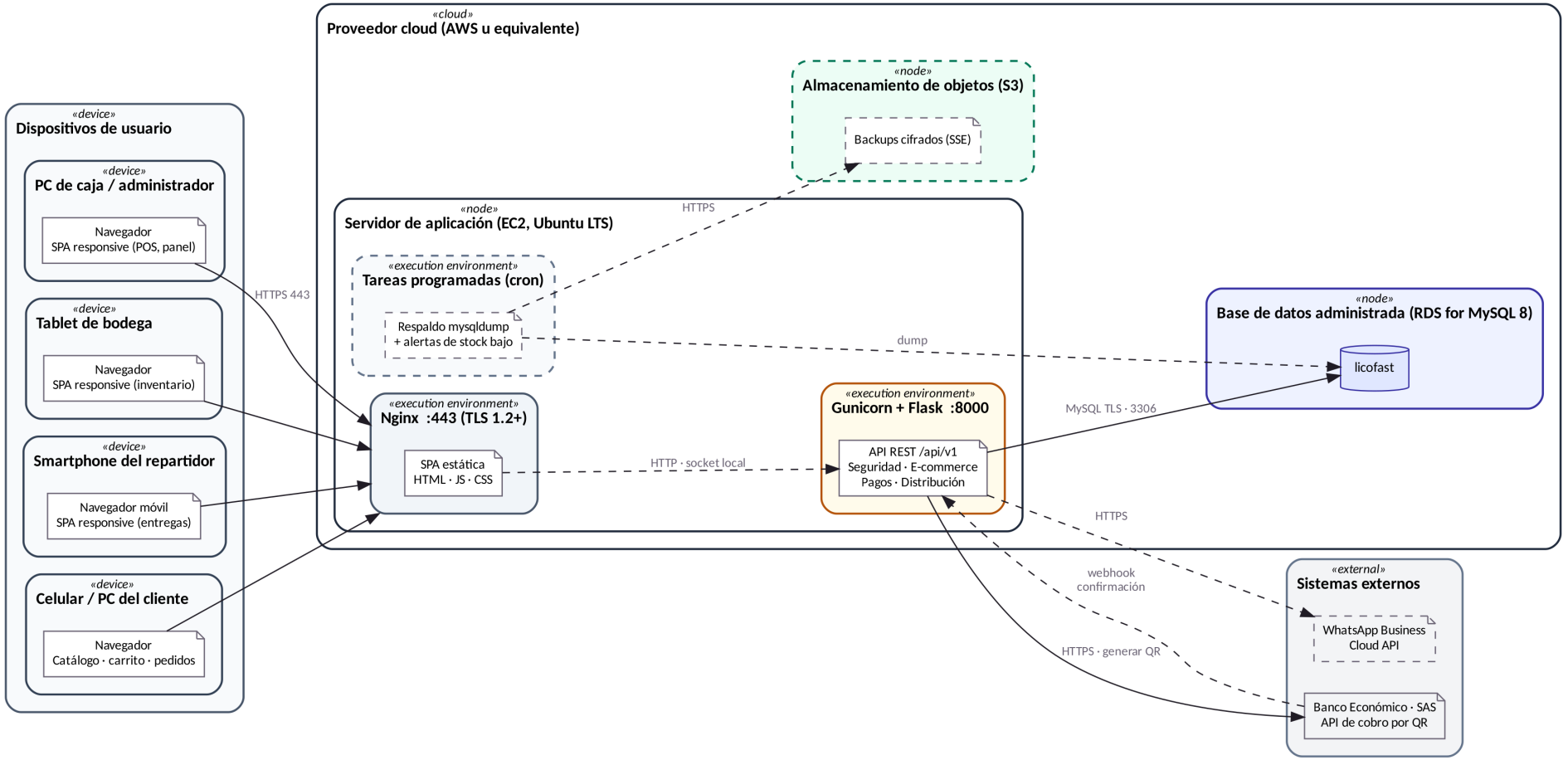


Figura 17. *Despliegue de producción en AWS con Nginx, Gunicorn, RDS y S3.*

8.5 Modelo entidad-relación

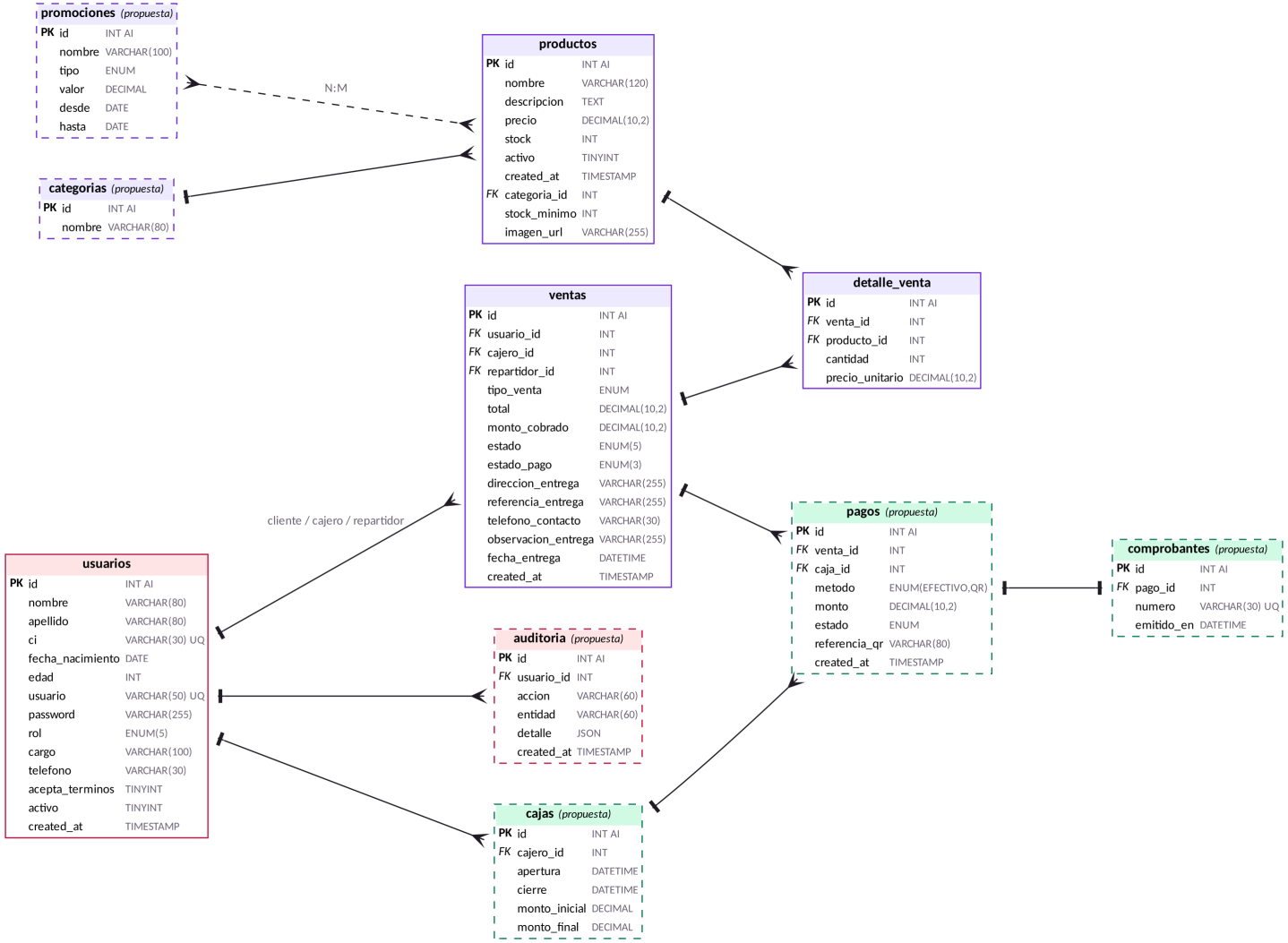


Figura 18. *Modelo entidad-relación: cuatro tablas existentes (trazo continuo) y nueve propuestas (punteado).*

8.6 Diagramas de secuencia

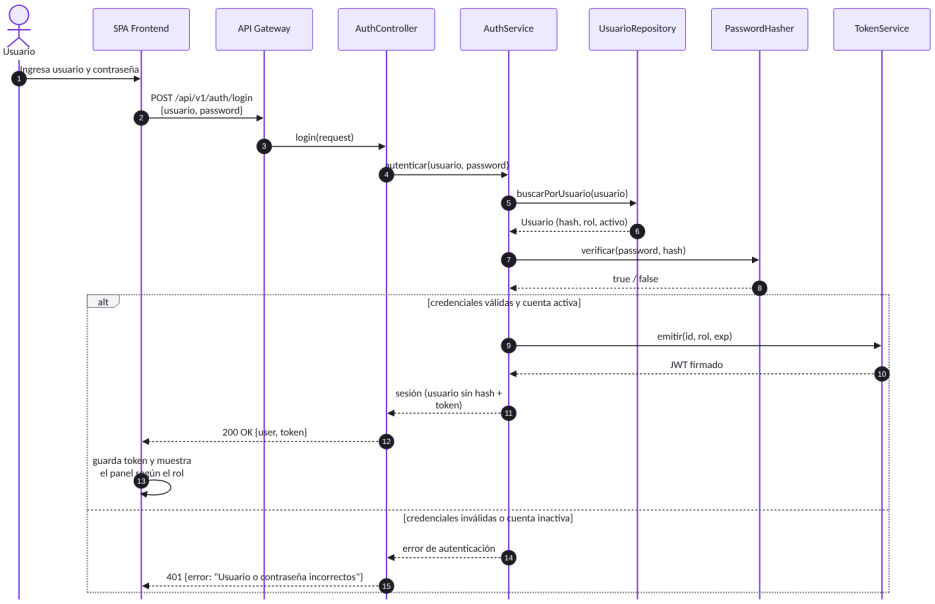


Figura 19. *Secuencia: inicio de sesión y emisión del JWT.*

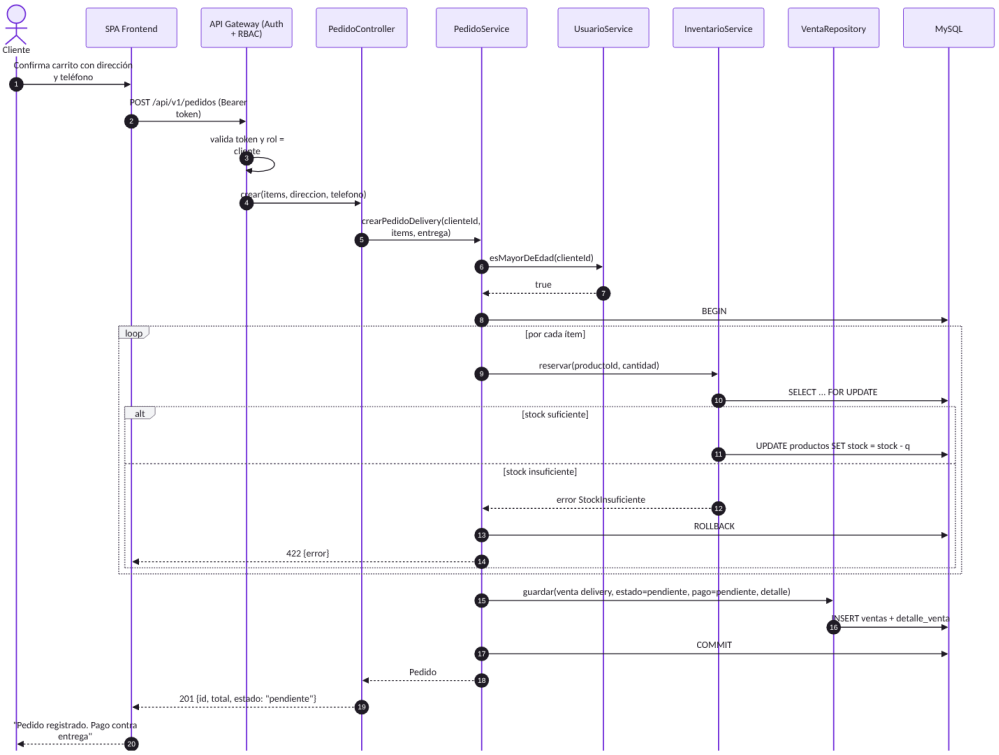


Figura 20. *Secuencia: checkout de un pedido de delivery.*

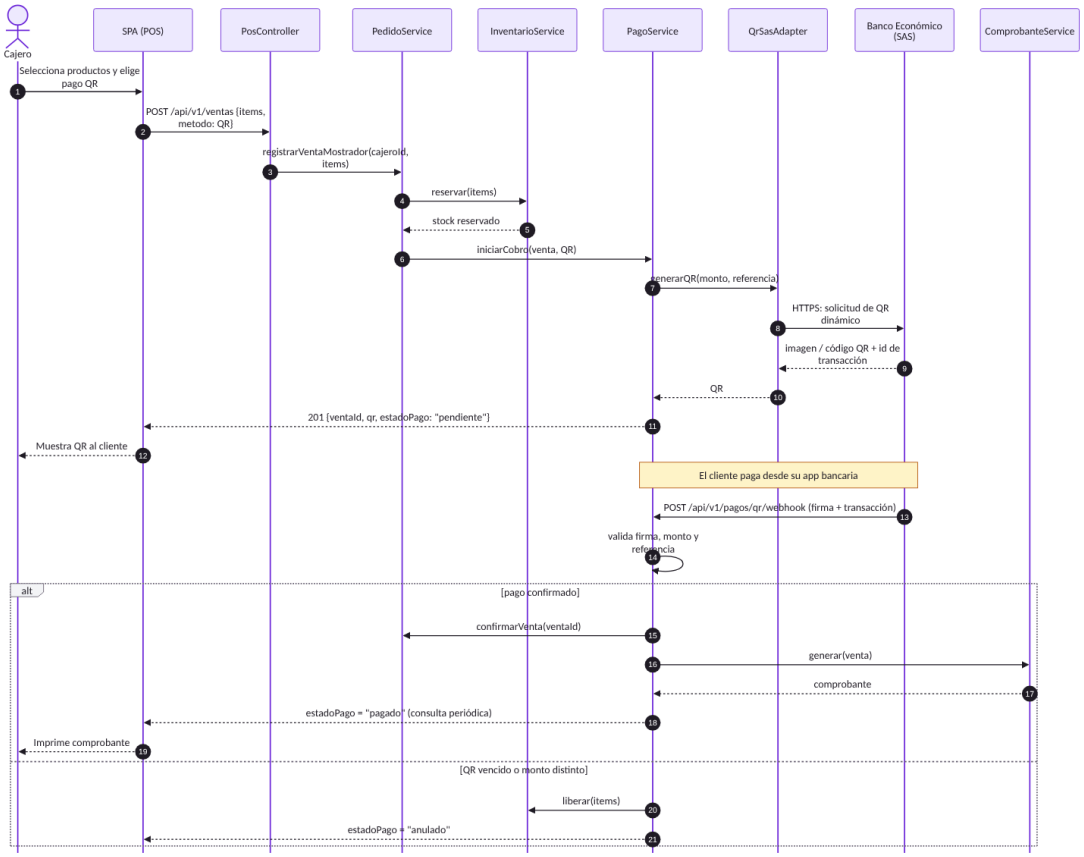


Figura 21. *Secuencia: venta en mostrador con pago por QR y confirmación por webhook.*

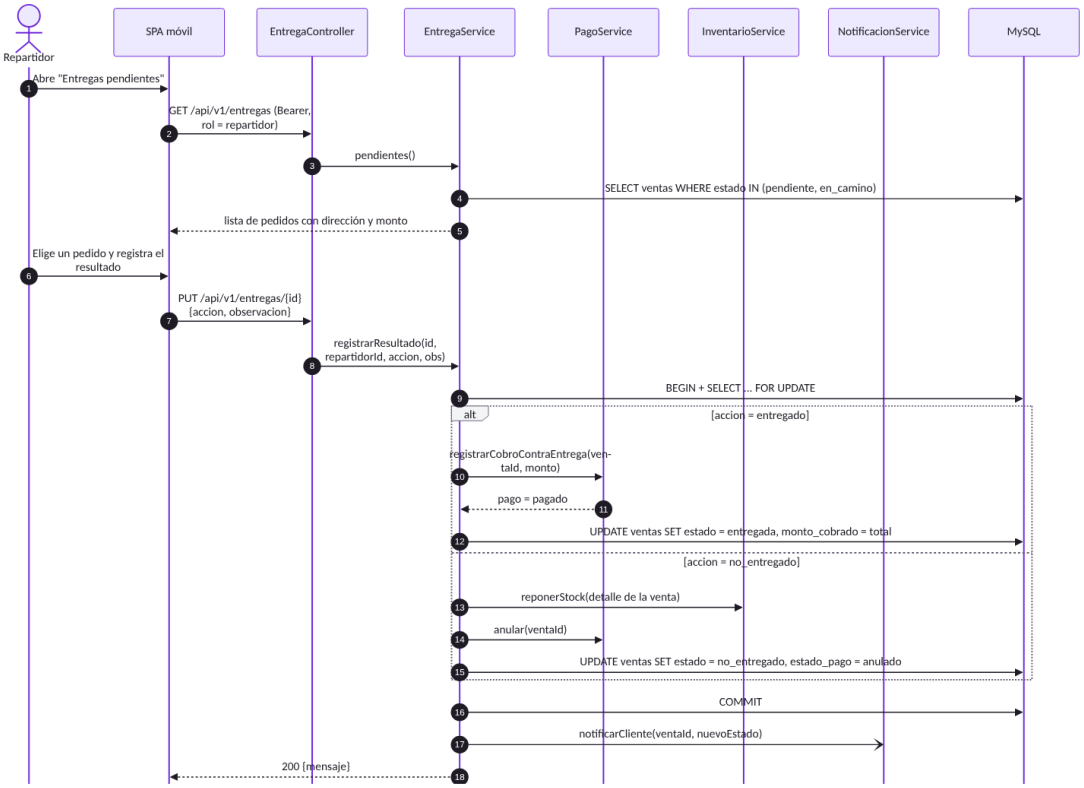


Figura 22. *Secuencia: cierre de una entrega (entregado o no entregado).*

8.7 Diagramas de estados

Ambos diagramas muestran las transiciones válidas; cualquier otra debe rechazarse con 409 (ver 7.13, patrón State).

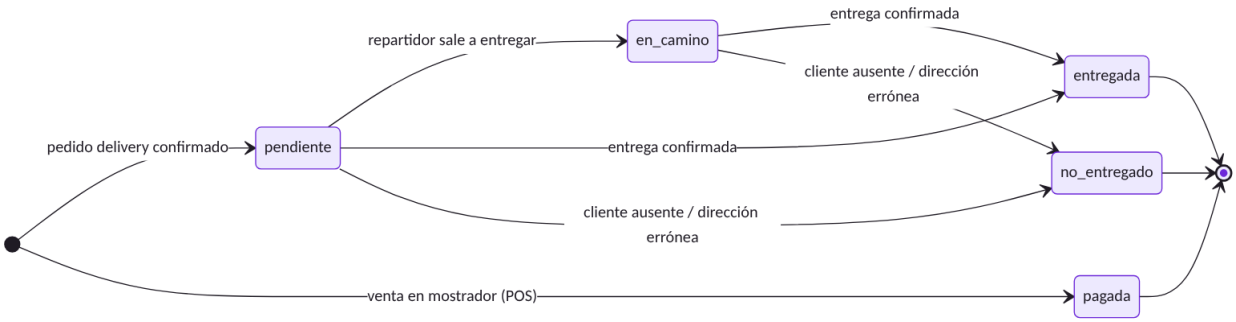


Figura 23. Estados de `ventas.estado`.

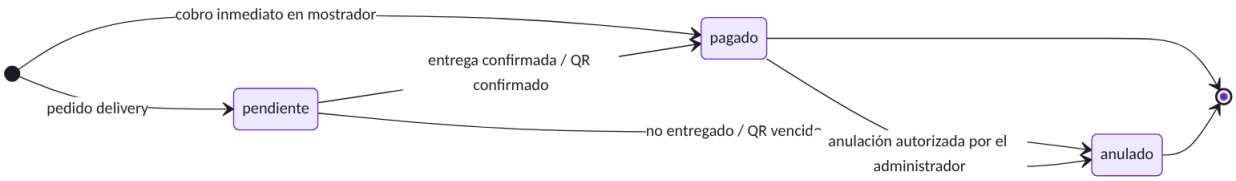


Figura 24. Estados de `ventas.estado_pago`.

9. Plan de implementación, pruebas y riesgos

9.1 Iteraciones

El plan lleva el prototipo actual hacia la arquitectura objetivo con entregas pequeñas y verificables. Cada iteración termina con algo que se puede demostrar. Las duraciones son estimaciones para un equipo de tres personas.

It.	Objetivo	Contenido	Hallazgos que cierra	Estimación
0	Base técnica	Repositorio único, variables de entorno, *pool*, CI con pruebas, formato de error, `/api/v1`.	H-05, H-11, H-13	1 semana
1	Seguridad	JWT firmado, RbacGuard con propiedad, límite de intentos, bloqueo, auditoría, CORS restringido, edad exacta.	H-01, H-04, H-06, H-07, H-09	2 semanas
2	E-commerce	Separación en capas, checkout con edad, *EstadoPedido*, inventario con movimientos, categorías, stock mínimo, caducidad de reservas.	H-02, H-08, H-10	3 semanas
3	Pagos básicos	Pago en efectivo, caja con apertura y cierre, comprobantes, anulación.	—	2 semanas
4	Distribución	Asignación, «en camino», cierre idempotente, historial y seguimiento para el cliente.	H-03	2 semanas
5	Pago QR	Adaptador SAS con simulador, webhook firmado e idempotente, vencimiento del QR.	—	2 semanas (sujeto al contrato del banco)
6	Puesta en marcha	Despliegue en AWS, Nginx con TLS, respaldos con prueba de restauración, WhatsApp, capacitación.	H-12 (retiro de duplicados)	2 semanas

9.2 Estrategia de pruebas

Nivel	Herramienta	Qué cubre	Criterio de aceptación
Unitarias	pytest	Reglas: edad, stock, transiciones de estado, hash de contraseña, cálculo de totales.	Cobertura ≥ 80 % en servicios
Integración	pytest con MySQL de pruebas	Repositorios y transacciones, incluida la concurrencia sobre el último producto.	Escenario EC-2 aprobado
Contrato de API	Postman y Newman	Códigos de estado, formato de error y permisos por rol (matriz 7.14).	Todos los endpoints con caso feliz y caso de rechazo
Seguridad	OWASP ZAP, revisión manual	Token manipulado, acceso a recursos ajenos, inyección, límite de intentos.	Sin hallazgos críticos ni altos
Pagos	Simulador del banco	QR vencido, monto distinto, notificación repetida, firma inválida.	Escenario EC-3 aprobado
Aceptación	Guion por historia de usuario	Los criterios de aceptación de la sección 4 con el cliente.	Historias de prioridad alta aprobadas
Carga	Locust	20 usuarios simultáneos en hora pico.	Escenario EC-4 aprobado

9.3 Riesgos

ID	Riesgo	Prob.	Impacto	Mitigación
R-1	El banco tarda en entregar el contrato técnico del QR.	Alta	Alto	Adaptador con simulador; lanzar primero con efectivo y contra entrega.
R-2	Vender a menores o sin trazabilidad de edad.	Media	Alto	Verificación en el servidor, auditoría y aceptación de términos.
R-3	Stock inconsistente por concurrencia.	Media	Alto	Bloqueo de fila, movimientos de stock y conciliación periódica.
R-4	Tiempo del equipo (tres personas, semestre).	Alta	Medio	Iteraciones cortas; el QR y WhatsApp quedan al final.
R-5	Mantener tres backends en paralelo.	Alta	Medio	ADR-02: una sola implementación de referencia.
R-6	Pérdida de datos.	Baja	Alto	Respaldo diario a S3 y prueba de restauración mensual.
R-7	Filtración de credenciales en el repositorio.	Media	Alto	Variables de entorno, `.gitignore` y rotación de claves.
R-8	Rechazo de los usuarios de caja o reparto (resistencia al cambio).	Media	Medio	Pantallas por rol con pocos pasos; capacitación breve.
R-9	Dependencia de WhatsApp Business (costo y aprobación).	Media	Bajo	Registro manual como alternativa (CU-E11).

9.4 Matriz de trazabilidad

Cada historia de usuario se conecta hacia arriba con los requisitos (qué se pide) y hacia abajo con el caso de uso y el componente que la realiza (cómo se construye). Permite comprobar que ningún requisito queda sin atender.

HU	Módulo	Requisitos	Caso de uso	Componente principal	Estado
HU-S01	Seguridad	RF-16, RF-17	CU-S01	AuthController, UsuarioService	Implementado
HU-S02	Seguridad	RF-18, RF-22	CU-S02	AuthService, JwtMiddleware	Implementado (sin bloqueo)
HU-S03	Seguridad	RF-18	CU-S03	AuthController	Implementado
HU-S04	Seguridad	RF-19	CU-S04	UsuarioController, UsuarioService	Implementado
HU-S05	Seguridad	RF-20, RF-22	CU-S05	UsuarioService, RbacGuard	Implementado
HU-S06	Seguridad	RF-25	CU-S06	AuditoriaService	Propuesta
HU-S07	Seguridad	RF-26	CU-S07	Tarea de respaldo (cron)	Propuesta
HU-E01	E-commerce	RF-01, RF-24	CU-E01	CatalogoService	Parcial
HU-E02	E-commerce	RF-02	CU-E02	CarritoService	Implementado
HU-E03	E-commerce	RF-02, RF-06, RF-16	CU-E03	PedidoService, InventarioService	Implementado (sin validar edad)
HU-E04	E-commerce	RF-09, RF-15, RF-18	CU-E04	PedidoController	Implementado
HU-E05	E-commerce	RF-03, RF-06	CU-E05	PedidoService	Implementado
HU-E06	E-commerce	RF-05, RF-24	CU-E06	CatalogoService	Parcial
HU-E07	E-commerce	RF-05, RF-23	CU-E07	InventarioService	Propuesta

HU	Módulo	Requisitos	Caso de uso	Componente principal	Estado
HU-E08	E-commerce	RF-07, RF-24	CU-E08	InventarioService	Parcial
HU-E09	E-commerce	RF-14, RF-18	CU-E09	PromocionService	Propuesta
HU-E10	E-commerce	RF-21	CU-E10	VentaRepository (reportes)	Implementado
HU-E11	E-commerce	RF-08	CU-E11	PedidoService	Propuesta
HU-P01	Pagos	RF-04	CU-P01	PagoService, PagoEfectivo	Implementado
HU-P02	Pagos	RF-04	CU-P02	PagoQr, QrSasAdapter, WebhookController	Propuesta (dividir en 2 historias)
HU-P03	Pagos	RF-04, RF-11	CU-P03	PagoService	Implementado
HU-P04	Pagos	RF-13	CU-P04	ComprobanteService	Propuesta
HU-P05	Pagos	RF-12	CU-P05, CU-P06	CajaService	Propuesta
HU-P06	Pagos	RF-04, RF-25	CU-P07	PagoService	Parcial
HU-D01	Distribución	RF-10, RF-11	CU-D01	EntregaController	Implementado
HU-D02	Distribución	RF-10	CU-D02	AsignacionService	Propuesta
HU-D03	Distribución	RF-09, RF-11	CU-D03	EntregaService	Propuesta
HU-D04	Distribución	RF-11	CU-D04	EntregaService	Implementado
HU-D05	Distribución	RF-06, RF-11	CU-D05	EntregaService	Implementado
HU-D06	Distribución	RF-11	CU-D06	EntregaController	Implementado
HU-D07	Distribución	RF-09	CU-D07, CU-D09	EntregaService, NotificacionService	Parcial

Cobertura
Los requisitos RF-01 a RF-26 están cubiertos por al menos una historia. Los no funcionales (RNF-01 a RNF-12) se verifican con los escenarios de la sección 7.15 y con las pruebas de la sección 9.2.

Anexo A. Glosario

Término	Significado
ADR	Registro de decisión arquitectónica (*Architecture Decision Record*).
Adapter	Patrón que traduce una interfaz externa a la que espera el sistema.
API REST	Interfaz web basada en recursos y verbos HTTP, sin estado.
Arqueo	Conteo del dinero de la caja contra lo que el sistema espera.
Cohesión	Grado en que los elementos de un módulo pertenecen juntos.
Acoplamiento	Grado de dependencia entre módulos.
CRUD	Crear, leer, modificar y eliminar.
Facade	Interfaz simple que oculta la complejidad interna de un módulo.
Idempotencia	Repetir una operación produce el mismo resultado que hacerla una vez.
JWT	Token firmado que lleva la identidad y el rol del usuario.
MVC	Separación entre modelo, vista y controlador.
POS	Punto de venta (*point of sale*), la caja del mostrador.
RBAC	Control de acceso basado en roles.
SAS	Servicio del Banco Económico para generar y confirmar pagos por código QR.
SDD	Documento de diseño de software.
Webhook	Llamada que un sistema externo hace al nuestro cuando ocurre un evento.
Baja lógica	Marcar un registro como inactivo en lugar de borrarlo.

Anexo B. Esquema de datos propuesto (DDL)

Cambios sobre `database/licofast.sql`. Las tablas existentes se conservan; se agregan columnas y nueve tablas.

Cambios en tablas existentes

```
ALTER TABLE productos
  ADD COLUMN categoria_id INT NULL,
  ADD COLUMN stock_minimo INT NOT NULL DEFAULT 5,
  ADD COLUMN imagen_url VARCHAR(255) NULL,
  ADD CONSTRAINT ck_stock CHECK (stock >= 0);
ALTER TABLE usuarios
  ADD COLUMN intentos_fallidos INT NOT NULL DEFAULT 0,
  ADD COLUMN bloqueado_hasta DATETIME NULL;
```

Catálogo y promociones

```
CREATE TABLE categorias (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(80) NOT NULL UNIQUE,
  activo TINYINT(1) NOT NULL DEFAULT 1);
ALTER TABLE productos
  ADD FOREIGN KEY (categoria_id) REFERENCES categorias(id);

CREATE TABLE promociones (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  tipo ENUM('porcentaje','monto') NOT NULL,
  valor DECIMAL(10,2) NOT NULL,
  producto_id INT NULL, categoria_id INT NULL,
  desde DATE NOT NULL, hasta DATE NOT NULL,
  activo TINYINT(1) NOT NULL DEFAULT 1,
  FOREIGN KEY (producto_id) REFERENCES productos(id),
  FOREIGN KEY (categoria_id) REFERENCES categorias(id));

CREATE TABLE movimientos_stock (
  id INT AUTO_INCREMENT PRIMARY KEY,
  producto_id INT NOT NULL,
  tipo ENUM('entrada','ajuste','venta','reposicion') NOT NULL,
  cantidad INT NOT NULL,
  motivo VARCHAR(200) NULL,
  venta_id INT NULL, usuario_id INT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (producto_id) REFERENCES productos(id),
  FOREIGN KEY (venta_id) REFERENCES ventas(id),
  FOREIGN KEY (usuario_id) REFERENCES usuarios(id));
```

Pagos, caja y comprobantes

```
CREATE TABLE cajas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  cajero_id INT NOT NULL,
  monto_inicial DECIMAL(10,2) NOT NULL,
  monto_esperado DECIMAL(10,2) NULL, monto_contado DECIMAL(10,2) NULL,
  abierta_en DATETIME NOT NULL, cerrada_en DATETIME NULL,
  FOREIGN KEY (cajero_id) REFERENCES usuarios(id));

CREATE TABLE pagos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  venta_id INT NOT NULL,
  metodo ENUM('efectivo','qr') NOT NULL,
  monto DECIMAL(10,2) NOT NULL,
  estado ENUM('pendiente','confirmado','vencido','anulado') NOT NULL,
  referencia VARCHAR(60) NULL UNIQUE,
  qr_expira_en DATETIME NULL, confirmado_en DATETIME NULL,
  FOREIGN KEY (venta_id) REFERENCES ventas(id));

CREATE TABLE movimientos_caja (
  id INT AUTO_INCREMENT PRIMARY KEY,
  caja_id INT NOT NULL, pago_id INT NULL,
  tipo ENUM('ingreso','egreso') NOT NULL,
  monto DECIMAL(10,2) NOT NULL, concepto VARCHAR(200) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (caja_id) REFERENCES cajas(id),
  FOREIGN KEY (pago_id) REFERENCES pagos(id));

CREATE TABLE comprobantes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  venta_id INT NOT NULL UNIQUE,
  numero VARCHAR(20) NOT NULL UNIQUE,
  emitido_en DATETIME NOT NULL,
  FOREIGN KEY (venta_id) REFERENCES ventas(id));
```

Auditoría y distribución

```
CREATE TABLE auditoria (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  usuario_id INT NULL,
  accion VARCHAR(60) NOT NULL, entidad VARCHAR(40) NOT NULL,
  entidad_id INT NULL, detalle JSON NULL, ip VARCHAR(45) NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (usuario_id) REFERENCES usuarios(id));

CREATE TABLE entregas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  venta_id INT NOT NULL, repartidor_id INT NOT NULL,
  resultado ENUM('entregado','no_entregado') NULL,
  observacion VARCHAR(255) NULL,
  asignada_en DATETIME NOT NULL, cerrada_en DATETIME NULL,
  FOREIGN KEY (venta_id) REFERENCES ventas(id),
  FOREIGN KEY (repartidor_id) REFERENCES usuarios(id));
```

Columnas y tablas adicionales de la implementación

La versión ejecutable (`LicoFastFinal/database/licofast.sql`) añade, además de lo anterior, lo necesario para los casos de uso completos:

- `ventas.canal` (web, mostrador, whatsapp) y `ventas.descuento` (total ahorrado por promociones).
- `movimientos_stock.tipo` incluye *inicial* (carga de stock al crear el producto), de modo que la suma de movimientos explica siempre el stock (invariante I-8).

- `pagos.id_transaccion`, `pagos.qr_payload` y `pagos.motivo_anulacion`.
- Tabla `idempotencia` (clave, usuario, estado y respuesta) para la cabecera *Idempotency-Key* de pedidos y pagos.

Anexo C. Del código actual a la arquitectura objetivo

Relación entre lo que hay en el repositorio y la capa o módulo donde debe quedar.

Ubicación actual	Contenido	Capa y módulo objetivo	Acción
`backend-python/app.py`	Rutas, autenticación, SQL y reglas en un archivo.	Capas 2 a 4, todos los módulos	Dividir en controladores, servicios y repositorios por módulo.
`backend-php/api/`	API equivalente en PHP.	Referencia de desarrollo	Mantener solo como prototipo; retirar al alcanzar la paridad.
`backend-java/`	Esqueleto Spring Boot.	Alternativa evaluada	No continuar (ADR-02).
`frontend/`	SPA con *fetch* a la API.	Capa 1	Apuntar a `/api/v1`; manejar el JWT.
`admin/`, `cajero/`, `cliente/`, `repartidor/`	Páginas PHP por rol.	Capa 1 (prototipo)	Sustituir por la SPA cuando exista paridad.
`config/`, `includes/`	Conexión, funciones de contraseña, plantillas.	Capas 3 y 4 (Seguridad)	Mover secretos a variables de entorno.
`database/licofast.sql`	Esquema y datos de prueba.	Capa 5	Aplicar el anexo B como migraciones; separar datos de prueba.
`LicoFastFinal/`	Implementación de referencia completa: API REST de 4 módulos en 5 capas, aplicación web por rol, esquema SQL y 30 pruebas.	Todas	Es el código que corresponde a este documento (anexo E).

Anexo D. Referencias

- Curso *Arquitectura de Software*: definición IEEE 1471, razones para una arquitectura, modelos de Shaw y Garlan, MVC, modelo de tres capas y modelo empresarial de cinco capas.
- Curso de *Ingeniería de Requisitos*: definición y especificación; validez, consistencia, completitud y realismo.
- Plantilla de casos de uso del material de clase (Letelier) y proceso de análisis: paquetes, clases entidad, interfaz y control.
- Visión del Proyecto LicoFast, Grupo N1.
- Repositorio del proyecto: github.com/FCCASTROCH/Licofast.
- IEEE Std 1471-2000, *Recommended Practice for Architectural Description of Software-Intensive Systems*.
- Shaw, M. y Garlan, D. *Software Architecture: Perspectives on an Emerging Discipline*. Prentice Hall, 1996.
- Fowler, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- Gamma, E. et al. *Design Patterns*. Addison-Wesley, 1994.
- OWASP Top 10 (2021) y OWASP ASVS: owasp.org.
- Fielding, R. *Architectural Styles and the Design of Network-based Software Architectures* (REST), 2000.
- RFC 7519, *JSON Web Token*; RFC 8018, *PKCS #5* (PBKDF2).

Anexo E. Implementación de referencia (LicoFastFinal)

La carpeta `LicoFastFinal` del repositorio contiene el código que realiza este diseño. Se ejecuta en local con XAMPP (MySQL) y Python; las instrucciones están en su README.

E.1 Estructura

Elemento	Contenido
`backend/app/core/`	JWT y PBKDF2 (*security*), guardas de rol (*auth*), errores uniformes, bus de eventos (Observer), idempotencia y validadores.
`backend/app/modules/seguridad/`	controller, service, repository y facade; respaldo con mysqldump.
`backend/app/modules/ecommerce/`	catálogo, promociones, inventario, pedidos, POS y reportes; *EstadoPedido* (State).
`backend/app/modules/pagos/`	PasarelaPago (Strategy), QrSasAdapter, caja, comprobantes y webhook firmado.
`backend/app/modules/distribucion/`	asignación, entregas, seguimiento y notificaciones (WhatsApp simulado).
`frontend/`	Aplicación web con las pantallas de los cinco roles.
`database/licofast.sql`	Esquema de 14 tablas y datos de demostración.
`backend/tests/`	30 pruebas automáticas de los escenarios de la sección 7.15.

E.2 Endpoints añadidos respecto a la sección 7

Además de los listados en 7.6 a 7.9, la implementación expone:

Módulo	Endpoint	Uso
Seguridad	POST /auth/logout · PUT /auth/me	Cerrar sesión (el token queda invalidado) y actualizar perfil o contraseña.
Seguridad	GET /auditoria · POST /admin/respaldo · GET /admin/respaldos	Bitácora con filtros y respaldos de la base.
E-commerce	GET, POST, PUT /categorias · GET, POST, PUT, DELETE /promociones	Mantenimiento de categorías y promociones.
E-commerce	POST /carrito/cotizar · POST /pedidos/whatsapp · GET /clientes	Totales con promociones; pedido por WhatsApp; búsqueda de clientes.
E-commerce	GET /inventario/movimientos · GET /inventario/alertas · GET /reportes/resumen	Trazabilidad de stock, stock bajo y panel del día.
Pagos	GET /pagos · POST /pagos/{id}/anular · POST /pagos/simulador/qr/{ref}/pagar	Cobranzas, anulación con motivo y simulador del banco (solo modo simulado).
Pagos	GET /cajas/actual · GET /cajas · POST /cajas/{id}/movimientos	Estado de la caja, arqueos y movimientos manuales.
Distribución	PUT /entregas/{id}/en-camino · GET /pedidos/{id}/seguimiento	Salida a entregar y línea de tiempo para el cliente.

E.3 Estado de los hallazgos de la sección 7.17

ID	Corrección en la implementación	Prueba
H-01	JWT HS256 con vencimiento e identificador; el cierre de sesión invalida el token.	EC-1, logout
H-02	El identificador del cliente sale del token; el campo enviado se ignora.	checkout
H-03	La transición valida el estado con bloqueo de fila; el segundo cierre responde 409 y no repone stock.	entrega doble
H-04	CORS limitado a una lista de orígenes; *debug* apagado por defecto; escucha solo en 127.0.0.1.	configuración
H-05	Credenciales y secretos por variables de entorno (`.env`). Parcial: los usuarios de demostración conservan una clave conocida y deben eliminarse o cambiarse en producción.	configuración
H-06	Crear un usuario exige contraseña; no existe valor por defecto.	usuario sin clave
H-07	Límite de intentos por IP y bloqueo de la cuenta tras 5 fallos (15 minutos).	bloqueo
H-08	Los pedidos de delivery sin atender se cancelan pasado el plazo y el stock vuelve.	pedido vencido
H-09	Edad calculada con la fecha exacta y verificada en el checkout y en la venta con cliente.	registro, menor
H-10	Baja lógica con historial; solo el administrador mantiene el catálogo.	RBAC, baja
H-11	Pool de conexiones, validación de entrada y formato de error uniforme sin detalles internos.	errores
H-12	Una sola implementación de referencia (Flask); los backends PHP y Java quedan como material histórico.	—
H-13	Rutas `/api/v1`, tabla de auditoría y columnas explícitas.	auditoría

Verificación
Resultado de las pruebas automáticas: 30 aprobadas de 30. Incluyen el último producto con cuatro compras simultáneas (EC-2), el webhook de pago repetido, con firma inválida y con monto incorrecto (EC-3) y el cierre doble de una entrega.